

Chapitre 8

Interagir avec l'interface utilisateur et l'état

v2.0

Objectifs du cours

- Poursuivez l'apprentissage des principes de base de Kotlin et continuez à créer des applications plus interactives.
 - Créez une application qui fonctionne avec les données saisies par l'utilisateur.
 - Découvrez comment utiliser l'état pour afficher les données et refléter automatiquement les modifications lorsqu'elles sont mises à jour.

Sommaire

- Présentation de l'état dans Compose
- Calculer un pourboire personnalisé
- Écrire des tests automatisés

Interagir avec l'interface utilisateur et l'état

Créez une application de calcul de pourboire qui calcule le montant d'un pourboire à partir des entrées utilisateur dans l'application.

Présentation de l'état dans Compose

- Cet atelier de programmation vous présente la notion **d'état** et la façon dont Jetpack Compose peut l'utiliser et le manipuler.
- En pratique, **on entend par "état" toute valeur pouvant évoluer au fil du temps dans une application.**
- Cette définition est très vaste et s'applique autant à une base de données qu'à une variable de votre application.
- Toutes les applications Android présentent des états à l'utilisateur.

Présentation de l'état dans Compose

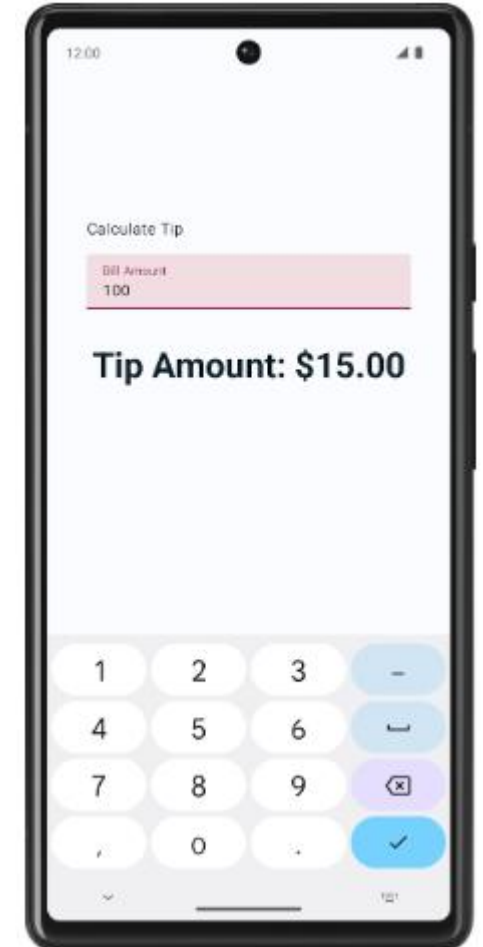
- Voici quelques exemples d'états que vous pouvez trouver dans les applications Android :
 - Message qui s'affiche quand une connexion réseau n'a pas pu être établie.
 - Formulaires, comme les formulaires d'inscription. L'état peut indiquer que le formulaire a été renseigné et envoyé.
 - Commandes tactiles telles que les boutons. L'état peut indiquer que le bouton n'a pas été utilisé, qu'il est en cours d'utilisation, comme sur un écran d'animation, ou qu'il a été utilisé, comme avec une action `onClick`.
- Dans cet atelier de programmation, vous découvrirez comment envisager les états et comment les exploiter lorsque vous utilisez Compose.

Présentation de l'état dans Compose

- Pour ce faire, vous créerez une application de calcul du pourboire appelée "Tip Time" à l'aide de ces éléments d'interface utilisateur Compose intégrés :
 - Un composable TextField pour saisir et modifier le texte.
 - Un composable Text pour afficher le texte.
 - Un composable Spacer pour afficher un espace vide entre les éléments d'interface utilisateur.
- À la fin de cet atelier de programmation, vous aurez créé un outil interactif de calcul de pourboire.
- Il déterminera automatiquement le montant du pourboire lorsque vous saisirez le coût du service.

Présentation de l'état dans Compose

- Cette image illustre la version finale de l'application :



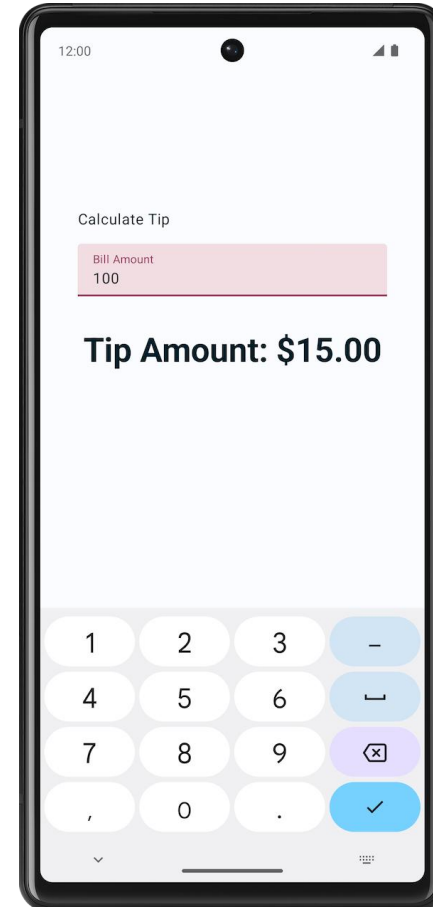
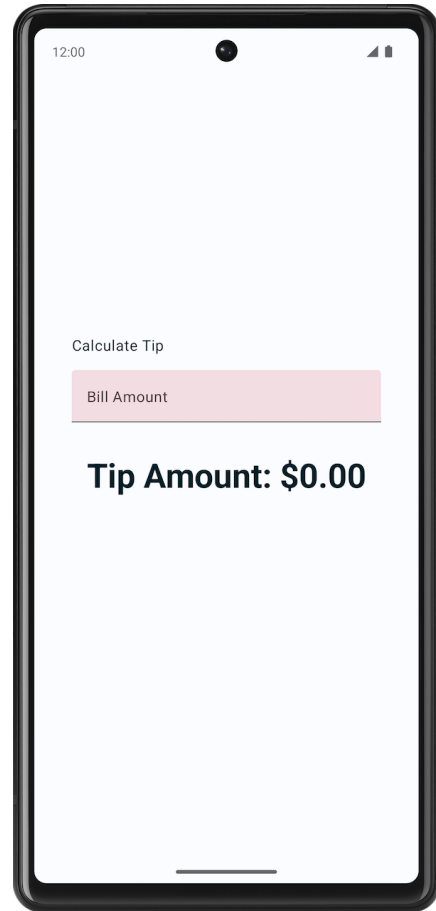
Présentation de l'état dans Compose

Premiers pas

- Dans ce parcours, vous créerez une application simple Android de calcul de pourboire.
- Les développeurs travaillent souvent de cette manière : ils préparent une version simple et opérationnelle de l'application (même si elle ne semble pas très esthétique), puis ils y ajoutent des fonctionnalités et la rendent plus agréable à l'œil par la suite.
- À la fin de cet atelier de programmation, votre application de calcul de pourboire ressemblera à ces captures d'écran.
- Lorsque l'utilisateur saisit un montant de facture, votre application affiche une suggestion de pourboire.

Présentation de l'état dans Compose

Premiers pas



Présentation de l'état dans Compose

Premiers pas

- Pour le moment, le pourcentage de pourboire est codé en dur sur une valeur correspondant à 15 %.
- Dans l'atelier de programmation suivant, vous continuerez à travailler sur votre application et ajouterez d'autres fonctionnalités, comme la personnalisation du pourcentage de pourboire.

Présentation de l'état dans Compose

Obtenir le code de démarrage

- Le code de démarrage est le code prédéfini qui peut servir de point de départ pour un nouveau projet.
- Il peut également vous aider à vous concentrer sur les nouveaux concepts présentés dans cet atelier de programmation.
- Le [code de démarrage](#) est sur dépôt GitHub.
Importer le projet dans Android Studio: File > Import Project.

Présentation de l'état dans Compose

Obtenir le code de démarrage

- **Présentation de l'application de démarrage**
- Pour vous familiariser avec le code de démarrage, procédez comme suit :
 - Dans Android Studio, ouvrez le projet contenant le code de démarrage.
 - **Exécutez l'application sur un appareil Android ou sur un émulateur.**
 - Deux composants de texte s'affichent. L'un sert simplement de libellé, et l'autre affiche le montant du pourboire.
- **Explication du code de démarrage**
- Le code de démarrage contient déjà les composables de texte.

Présentation de l'état dans Compose

Obtenir le code de démarrage

- Dans cet atelier, vous allez ajouter un champ de texte pour recevoir les entrées des utilisateurs.
- Voici une présentation rapide de certains fichiers pour vous aider à vous lancer.
- **res > values > strings.xml**
 - `<resources>`
 - `<string name="app_name">Tip Time</string>`
 - `<string name="calculate_tip">Calculate Tip</string>`
 - `<string name="bill_amount">Bill Amount</string>`
 - `<string name="tip_amount">Tip Amount: %s</string>`
 - `</resources>`

Présentation de l'état dans Compose

Obtenir le code de démarrage

- Il s'agit du fichier string.xml dans les ressources.
- Il contient toutes les chaînes que vous utiliserez dans cette application.
- **MainActivity**
- Ce fichier contient principalement du code généré à partir du modèle, ainsi que les fonctions suivantes :
 - La fonction TipTimeLayout() contient un élément Column avec les deux composables de texte que vous voyez dans les captures d'écran.
 - Un composable spacer permet également d'ajouter de l'espace pour des raisons esthétiques.

Présentation de l'état dans Compose

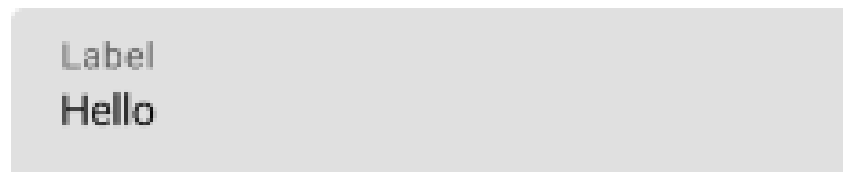
Obtenir le code de démarrage

- La fonction `calculateTip()` reçoit le montant de la facture et calcule un pourboire de 15 %.
- Le paramètre `tipPercent` est défini sur une valeur d'argument par défaut de 15.0.
- Par conséquent, le pourboire par défaut est actuellement de 15 %.
- Dans l'atelier de programmation suivant, le montant du pourboire dépendra du choix de l'utilisateur.
- Dans le bloc `Surface()` de la fonction `onCreate()`, la fonction `TipTimeLayout()` est appelée.
- Dans le bloc `TipTimeTheme` de la fonction `TipTimeLayoutPreview()`, la fonction `TipTimeLayout()` est appelée.

Présentation de l'état dans Compose

Obtenir le code de démarrage

- **Recevoir les entrées de l'utilisateur**
- Dans cette section, vous ajouterez l'élément d'interface qui permettra à l'utilisateur de saisir le montant de la facture dans l'application, comme illustré sur l'image ci-dessous :



Présentation de l'état dans Compose

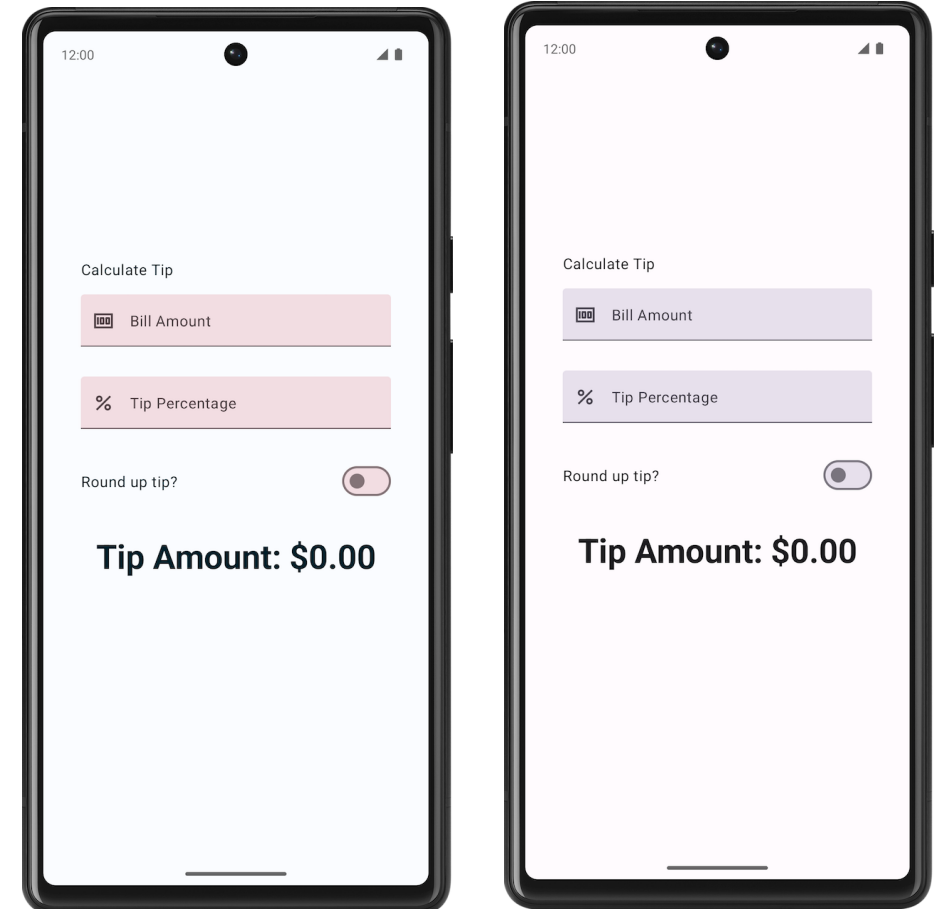
Obtenir le code de démarrage

- Votre application utilise un **style** et un **thème** personnalisés.
- **Les styles et les thèmes sont des ensembles d'attributs qui spécifient l'apparence d'un seul élément d'interface utilisateur.**
- Un style peut par exemple spécifier la couleur de la police, sa taille et sa couleur d'arrière-plan, et ces valeurs seront appliquées à l'ensemble de l'application.
- Dans d'autres ateliers de programmation, nous verrons comment implémenter ces éléments dans votre application.
- Pour l'instant, cela a déjà été fait afin d'obtenir un rendu plus esthétique.

Présentation de l'état dans Compose

Obtenir le code de démarrage

- Pour mieux visualiser la différence, voici une comparaison côte à côte du rendu final de l'application avec et sans thème personnalisé.



Présentation de l'état dans Compose

Obtenir le code de démarrage

- Remarque : lorsque vous ajoutez un composable de champ de texte dans votre application, celui-ci utilise un jeu de couleurs qui dépend du thème personnalisé.
- La fonction composable **TextField** permet à l'utilisateur de saisir du texte dans une application.
- Ajoutez le composable TextField à l'application :
 - Dans le fichier MainActivity.kt, ajoutez une fonction composable EditNumberField() qui accepte un paramètre Modifier.
 - Dans le corps de la fonction EditNumberField(), ajoutez un élément TextField qui accepte un paramètre nommé value défini sur une chaîne vide et un paramètre nommé onValueChange défini sur une expression lambda vide :

Présentation de l'état dans Compose

Obtenir le code de démarrage

- `@Composable`
- `fun EditNumberField(modifier: Modifier = Modifier) {`
- `TextField(`
- `value = "",`
- `onChange = {},`
- `modifier = modifier`
- `)`
- `}`
- Notez les paramètres que vous avez transmis :
 - Le paramètre **value** est une zone de texte qui affiche la valeur de chaîne que vous transmettez ici.

Présentation de l'état dans Compose

Obtenir le code de démarrage

- Le paramètre **onValueChange** est le rappel lambda qui est déclenché lorsque l'utilisateur saisit du texte dans la zone à cet effet.
- Importez cette fonction :
 - `import androidx.compose.material3.TextField`
- Dans le composable `TipTimeLayout()`, sur la ligne située après la première fonction composable de texte, appelez la fonction `EditNumberField()` en transmettant le modificateur:
 - `import androidx.compose.foundation.layout.fillMaxWidth`
 - `@Composable`
 - `fun TipTimeLayout() {`

Présentation de l'état dans Compose

Obtenir le code de démarrage

- `Column(`
- `modifier = Modifier`
- `.statusBarsPadding()`
- `.padding(horizontal = 40.dp)`
- `.verticalScroll(rememberScrollState())`
- `.safeDrawingPadding(),`
- `horizontalAlignment = Alignment.CenterHorizontally,`
- `verticalArrangement = Arrangement.Center`
- `) {`
- `Text(`
- `...`

Présentation de l'état dans Compose

Obtenir le code de démarrage

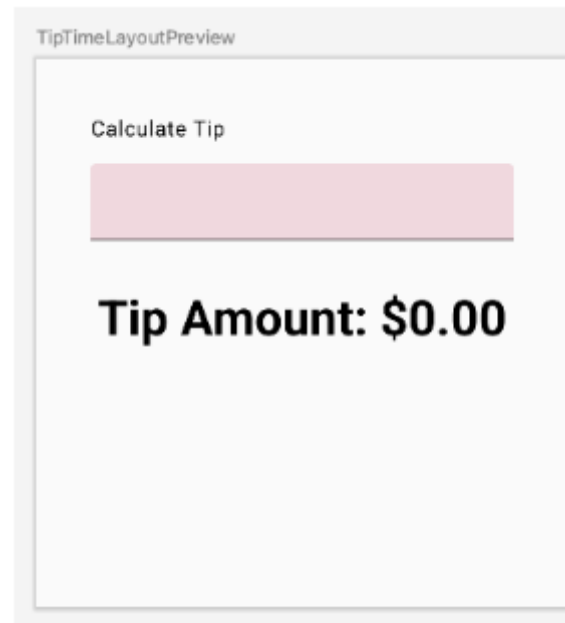
- `)`
- `EditNumberField(modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth())`
- `Text(`
- `...`
- `)`
- `...`
- `}`
- `}`

- La zone de texte s'affiche à l'écran.

Présentation de l'état dans Compose

Obtenir le code de démarrage

- Dans le volet Design (Conception), vous devriez voir le texte Calculate Tip, une zone de texte vide et le composable de texte Tip Amount.



Présentation de l'état dans Compose

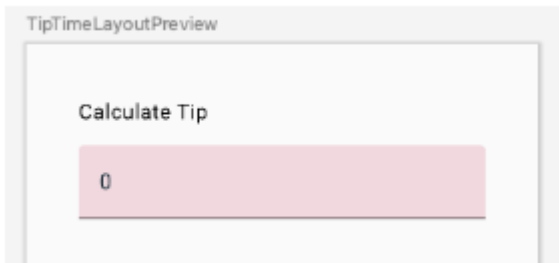
Utiliser l'état dans Compose

- **Dans une application, l'état correspond à toute valeur susceptible de changer au fil du temps.**
- Dans cette activité, il s'agit du montant de la facture.
- Ajoutez une variable pour stocker cet état :
 - Au début de la fonction `EditNumberField()`, utilisez le mot clé `val` pour ajouter une variable `amountInput` et lui attribuer la valeur `"0"` :
 - `val amountInput = "0"`
- Il s'agit de l'état de l'application correspondant au montant de la facture.

Présentation de l'état dans Compose

Utiliser l'état dans Compose

- Définissez le paramètre nommé `value` sur une valeur `amountInput` :
 - `TextField(`
 - `value = amountInput,`
 - `onValueChange = {},`
 - `)`
- Vérifiez l'aperçu.
 - La zone de texte affiche la valeur définie pour la variable d'état, comme illustré sur cette image :



Présentation de l'état dans Compose

Utiliser l'état dans Compose

- Exécutez l'application dans l'émulateur et essayez de saisir une autre valeur.
- L'état codé en dur reste le même, car le composable TextField ne se met pas à jour tout seul.
- Il se met à jour lorsque son paramètre value change, lequel est défini sur la propriété amountInput.
- La variable amountInput représente l'état de la zone de texte.
- **Un état codé en dur n'est pas utile**, car il ne peut pas être modifié et ne reflète pas l'entrée utilisateur.

Présentation de l'état dans Compose

Utiliser l'état dans Compose

- Vous devez mettre à jour l'état de l'application lorsque l'utilisateur modifie le montant de la facture.

Présentation de l'état dans Compose

Composition

- Les composables d'une application décrivent une interface utilisateur qui affiche une colonne de texte, un espace vide et une zone de texte.
- Le texte affiche un élément Calculate Tip, et la zone de texte affiche une valeur 0 ou toute valeur par défaut.
- **Compose est un framework d'interface utilisateur déclaratif**, ce qui signifie que vous déclarez la façon dont l'UI doit apparaître dans le code.
- Si vous souhaitez que la zone de texte affiche initialement la valeur 100, vous devez définir la valeur 100 comme valeur initiale dans le code des composables.

Présentation de l'état dans Compose

Composition

- Que se passe-t-il si vous souhaitez que votre interface utilisateur change pendant que l'application est en cours d'exécution ou lorsque l'utilisateur interagit avec elle ?
- Par exemple, si vous souhaitez remplacer la variable `amountInput` par la valeur saisie par l'utilisateur et l'afficher dans la zone de texte ?
- C'est là qu'intervient un processus appelé **recomposition**, qui **permet de mettre à jour la composition de l'application**.
- Le champ `Composition` est une description de l'interface utilisateur créée par Compose lorsqu'il exécute des composables.

Présentation de l'état dans Compose

Composition

- Les applications Compose appellent des fonctions modulables pour convertir les données en UI.
- **En cas de changement d'état, Compose réexécute les fonctions modulables concernées avec le nouvel état, ce qui crée une UI mise à jour.**
- **Ce processus est appelé recomposition.**
- **Compose planifie une recomposition pour vous.**
- Lorsque Compose exécute vos composables pour la première fois lors de la composition initiale, il suit les composables que vous appelez pour décrire votre UI dans une composition.

Présentation de l'état dans Compose

Composition

- **La recomposition désigne le moment où Compose réexécute les composables qui ont pu changer en réponse à des modifications de données, puis met à jour la composition pour refléter ces modifications.**
- La composition ne peut être produite que par une composition initiale et mise à jour par une recomposition.
- **Le seul moyen de modifier la composition est la recomposition.**
- **Pour ce faire, Compose doit connaître l'état à suivre.**
- Ainsi, lorsqu'il reçoit une mise à jour, il peut planifier la recomposition.

Présentation de l'état dans Compose

Composition

- Dans votre cas, il s'agit de la variable `amountInput`.
- Chaque fois que sa valeur changera, Compose programmera une recomposition.
- **Vous pouvez utiliser les types `State` et `MutableState` dans Compose pour rendre l'état observable ou suivi par Compose.**
- Le type `State` est immuable.
- Vous ne pouvez donc que lire sa valeur, tandis que le type `MutableState` est modifiable.
- Vous pouvez utiliser la fonction `mutableStateOf()` pour créer un élément `MutableState` observable.

Présentation de l'état dans Compose

Composition

- Elle reçoit une valeur initiale en tant que paramètre encapsulé dans un objet State, ce qui rend son élément value observable.
- La valeur renvoyée par la fonction mutableStateOf() :
 - contient l'état, qui correspond au montant de la facture ;
 - est modifiable, de sorte que sa valeur peut être ajustée ;
 - est observable, de sorte que Compose observe toute modification apportée à la valeur et déclenche une recomposition pour mettre à jour l'UI.
- Ajoutez un état de coût de service :
 - Dans la fonction EditNumberField(), remplacez le mot clé val avant la variable d'état amountInput par var :

Présentation de l'état dans Compose Composition

- `var amountInput = "0"`
- Ce changement rend `amountInput` modifiable.
- Utilisez le type **MutableState<String>** au lieu de la variable `String` codée en dur pour que Compose sache qu'il doit suivre l'état `amountInput`, puis transmettre une chaîne `"0"`, qui correspond à la valeur par défaut initiale de la variable d'état `amountInput` :
 - `import androidx.compose.runtime.MutableState`
 - `import androidx.compose.runtime.mutableStateOf`
 - `var amountInput: MutableState<String> = mutableStateOf("0")`

Présentation de l'état dans Compose

Composition

- L'initialisation de `amountInput` peut également être écrite comme suit avec l'inférence de type :
 - `var amountInput = mutableStateOf("0")`
- La fonction `mutableStateOf()` reçoit une valeur "0" initiale en tant qu'argument, ce qui rend `amountInput` observable.
- Un avertissement de compilation est alors généré dans Android Studio, mais vous résoudrez bientôt ce problème :
 - `Creating a state object during composition without using remember.`
- Dans la fonction composable `TextField`, utilisez la propriété `amountInput.value` :

Présentation de l'état dans Compose

Composition

- `TextField(`
- `value = amountInput.value,`
- `onValueChange = {},`
- `modifier = modifier`
- `)`
- Compose effectue le suivi de chaque composable qui lit les propriétés d'état `value` et déclenche une recomposition lorsque son élément `value` change.
- Le rappel `onValueChange` est déclenché lorsque l'entrée de la zone de texte change. Dans l'expression lambda, la variable `it` contient la nouvelle valeur.

Présentation de l'état dans Compose

Composition

- Dans l'expression lambda du paramètre nommé `onValueChange`, définissez la propriété `amountInput.value` sur la variable `it` :
 - `@Composable`
 - `fun EditNumberField(modifier: Modifier = Modifier) {`
 - `var amountInput = mutableStateOf("0")`
 - `TextField(`
 - `value = amountInput.value,`
 - `onValueChange = { amountInput.value = it },`
 - `modifier = modifier`
 - `)`
 - `}`

Présentation de l'état dans Compose

Composition

- Vous mettez à jour l'état le TextField (à savoir, la variable `amountInput`) lorsque TextField vous informe d'une modification du texte via la fonction de rappel `onValueChange`.
- Exécutez l'application et saisissez du texte dans la zone de texte.
- La zone de texte affiche toujours une valeur 0.
- Lorsque l'utilisateur saisit du texte dans la zone à cet effet, le rappel `onValueChange` est appelé et la variable `amountInput` est remplacée par la nouvelle valeur.
- L'état `amountInput` est suivi par Compose.

Présentation de l'état dans Compose

Composition

- Dès lors, lorsque sa valeur changera, la recomposition sera planifiée, et la fonction modifiable `EditNumberField()` sera réexécutée.
- Dans cette fonction modifiable, la variable `amountInput` est réinitialisée et revient donc à sa valeur 0 initiale.
- Ainsi, la zone de texte affiche une valeur 0.
- Avec le code que vous avez ajouté, les changements d'état entraînent la planification des recompositions.

Présentation de l'état dans Compose

Composition

- Cependant, vous devez conserver la valeur de la variable `amountInput` lors des recompositions pour qu'elle ne soit pas réinitialisée et remplacée par une valeur 0 chaque fois que la fonction `EditNumberField()` effectue une recomposition.
- Vous résoudrez ce problème dans la section suivante.

Présentation de l'état dans Compose

Utiliser la fonction remember

- Les méthodes de composables peuvent être appelées plusieurs fois en raison de la recomposition.
- **Le composable réinitialise son état lors de la recomposition s'il n'est pas enregistré.**
- Les fonctions modulables peuvent stocker un objet lors des recompositions à l'aide de la fonction **remember**.
- **Une valeur calculée par la fonction remember est stockée dans la composition lors de la composition initiale, et la valeur stockée est renvoyée lors de la recomposition.**

Présentation de l'état dans Compose

Utiliser la fonction remember

- **Les fonctions remember et mutableStateOf sont généralement utilisées conjointement dans des fonctions modulables pour que l'état et ses mises à jour soient correctement reflétés dans l'interface utilisateur.**
- Utilisez la fonction remember dans la fonction EditNumberField() :
 - Dans la fonction EditNumberField(), initialisez la variable amountInput avec le délégué de propriété Kotlin remember de by en encadrant l'appel de la fonction mutableStateOf() avec remember.
 - Dans la fonction mutableStateOf(), transmettez une chaîne vide au lieu d'une chaîne "0" statique :

Présentation de l'état dans Compose

Utiliser la fonction remember

- `var amountInput by remember { mutableStateOf("") }`
- La chaîne vide correspond désormais à la valeur par défaut initiale de la variable `amountInput`.
- **by** est une délégation de propriété de Kotlin.
- Les fonctions "getter" et "setter" par défaut de la propriété `amountInput` sont déléguées aux fonctions "getter" et "setter" de la classe `remember`, respectivement.
- Importez ces fonctions :
 - `import androidx.compose.runtime.remember`
 - `import androidx.compose.runtime.getValue`
 - `import androidx.compose.runtime.setValue`

Présentation de l'état dans Compose

Utiliser la fonction remember

- Avertissement : Ce code peut générer l'erreur si tel est le cas, ajoutez manuellement les importations `getValue` et `setValue` dans le bloc d'importation au début du fichier, comme illustré sur cette image :

```
import androidx.compose.material.*
import androidx.compose.material.Surface
import androidx.compose.material.Text
import androidx.compose.material.TextField
import androidx.compose.runtime.Composable
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
```

Présentation de l'état dans Compose

Utiliser la fonction remember

- L'ajout des importations "getter" et "setter" du délégué vous permet de lire et de définir amountInput sans faire référence à la propriété value de MutableState.
- La fonction EditNumberField() mise à jour devrait se présenter comme suit :
 - @Composable
 - fun EditNumberField(modifier: Modifier = Modifier) {
 - var amountInput by remember { mutableStateOf("") }
 - TextField(
 - value = amountInput,
 - onValueChange = { amountInput = it },

Présentation de l'état dans Compose

Utiliser la fonction remember

- `modifier = modifier`
 - `)`
 - `}`
- Remarque : Lors de la composition initiale, value dans TextField est défini sur la valeur initiale, qui est une chaîne vide.
 - Lorsque l'utilisateur saisit du texte dans le champ de texte, le rappel lambda `onValueChange` est appelé, la fonction lambda s'exécute, et la valeur `amountInput.value` est définie sur la valeur mise à jour dans le champ de texte.

Présentation de l'état dans Compose

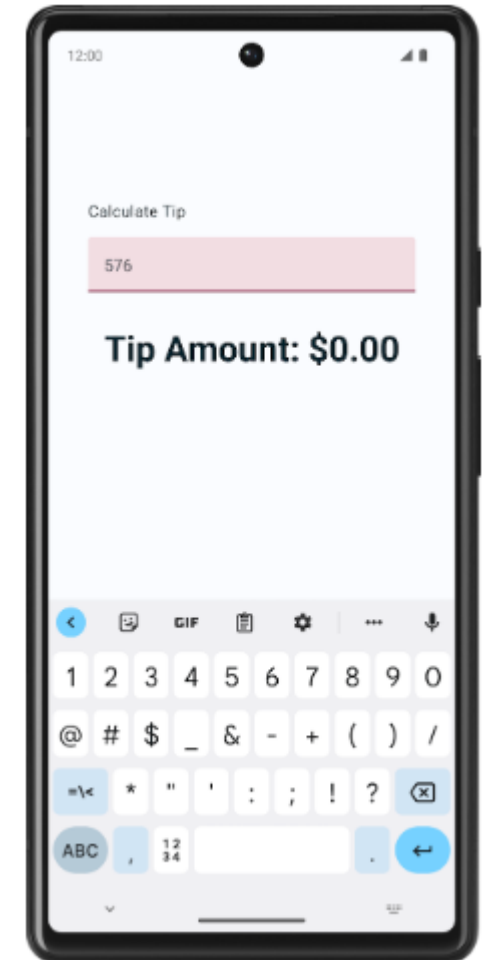
Utiliser la fonction remember

- amountInput correspond à l'état modifiable suivi par Compose.
- La recomposition est planifiée.
- La fonction modulable EditNumberField() est recomposée.
- Comme vous utilisez remember { }, la modification survit à la recomposition.
- C'est pourquoi l'état n'est pas réinitialisé et remplacé par "".
- L'élément value du champ de texte est défini sur la valeur mémorisée pour amountInput.
- Le champ de texte se recompose (comme illustré à l'écran avec une nouvelle valeur).

Présentation de l'état dans Compose

Utiliser la fonction remember

- Exécutez l'application et saisissez du texte dans la zone à cet effet.
- Le texte que vous avez saisi devrait maintenant s'afficher.



Présentation de l'état dans Compose

Changement d'état et recomposition à l'œuvre

- Dans cette section, vous définirez un point d'arrêt et déboguerez la fonction modifiable `EditNumberField()` pour voir comment fonctionnent la composition et la recomposition initiales.
- Remarque : Un point d'arrêt correspond à l'endroit dans le code où vous souhaitez interrompre l'exécution normale de votre application pour effectuer d'autres actions (par exemple, examiner des variables, évaluer des expressions ou exécuter le code ligne par ligne). Vous pouvez définir un point d'arrêt au niveau de n'importe quelle ligne de code exécutable.
- Définissez un point d'arrêt et déboguez l'application sur un émulateur ou un appareil :

Présentation de l'état dans Compose

Changement d'état et recomposition à l'œuvre

- Dans la fonction `EditNumberField()`, à côté du paramètre nommé `onValueChange`, définissez un point d'arrêt de ligne.
- Dans le menu de navigation, cliquez sur Déboguer l'application.
- L'application se lance sur l'émulateur ou l'appareil.
- L'exécution de votre application s'interrompt pour la première fois lorsque l'élément `TextField` est créé.
- Dans le volet Debug (Débogage), cliquez sur Resume Program (Reprendre le programme).
- La zone de texte est créée.

Présentation de l'état dans Compose

Changement d'état et recomposition à l'œuvre

- Sur l'émulateur ou l'appareil, saisissez une lettre dans la zone de texte.
- L'exécution de votre application s'interrompt de nouveau lorsqu'elle atteint le point d'arrêt que vous avez défini.
- Lorsque vous saisissez le texte, le rappel `onValueChange` est appelé.
- Dans le lambda `it`, vous avez la nouvelle valeur que vous avez saisie au clavier.
- Une fois que la valeur `"it"` est attribuée à `amountInput`, Compose déclenche la recomposition avec les nouvelles informations, étant donné que la valeur observable a changé.

Présentation de l'état dans Compose

Changement d'état et recomposition à l'œuvre

```
@Composable
fun EditNumberField() {
    var amountInput by remember { mutableStateOf( value: "" ) }
    TextField(
        value = amountInput,
        onChange = { amountInput = it }, amountInput$de
    )
}
```

- Dans le volet Debug (Débogage), cliquez sur Resume Program (Reprendre le programme).
- Le texte saisi dans l'émulateur ou sur l'appareil s'affiche à côté de la ligne avec le point d'arrêt.

Présentation de l'état dans Compose

Changement d'état et recomposition à l'œuvre

- Il s'agit de l'état du champ de texte.
- Cliquez sur Resume Program (Reprendre le programme).
- La valeur saisie s'affiche sur l'émulateur ou l'appareil.

Présentation de l'état dans Compose

Modifier l'apparence

- Dans la section précédente, vous avez rendu le champ de texte fonctionnel.
- Dans cette section, vous améliorerez l'interface utilisateur.

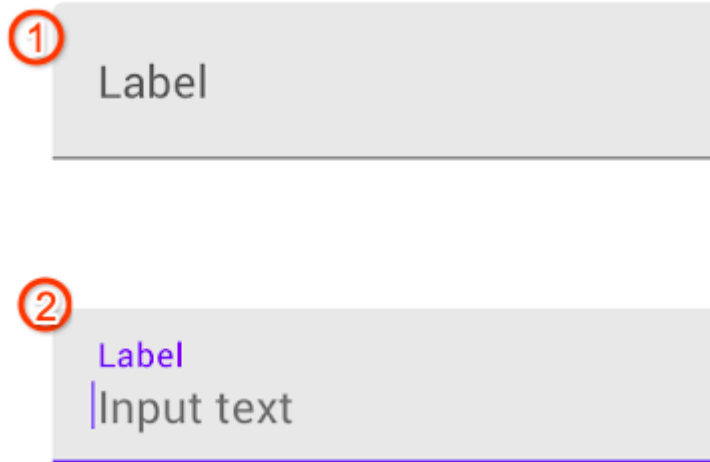
Présentation de l'état dans Compose

Modifier l'apparence

- **Ajouter un libellé à la zone de texte**
- **Chaque zone de texte doit comporter un libellé indiquant aux utilisateurs les informations qu'ils peuvent saisir.**
- Dans la première partie de l'exemple d'image suivant, le libellé se trouve au milieu d'un champ de texte et est aligné avec la ligne d'entrée.
- Dans la deuxième partie de l'exemple d'image suivant, le libellé se place plus haut dans la zone de texte lorsque l'utilisateur clique dans la zone pour saisir du texte.

Présentation de l'état dans Compose

Modifier l'apparence



Présentation de l'état dans Compose

Modifier l'apparence

- Pour en savoir plus sur l'anatomie des champs de texte, consultez [Anatomie](#).
- Modifiez la fonction `EditNumberField()` pour ajouter un libellé au champ de texte :
 - Dans la fonction composable `TextField()` de la fonction `EditNumberField()`, ajoutez un ensemble de paramètres `label` à une expression lambda vide :
 - Dans l'expression lambda, appelez la fonction `Text()` qui accepte une ressource `stringResource(R.string.bill_amount)` :
 - `label = { Text(stringResource(R.string.bill_amount)) },`

Présentation de l'état dans Compose

Modifier l'apparence

- Dans la fonction composable `TextField()`, ajoutez un paramètre nommé `singleLine` défini sur la valeur `true` :
 - `TextField(`
 - `// ...`
 - `singleLine = true,`
 - `)`
- La zone de texte est alors condensée en une seule ligne à faire défiler horizontalement.
- Ajoutez le paramètre `keyboardOptions` défini sur `KeyboardOptions()` :
 - `import androidx.compose.foundation.text.KeyboardOptions`

Présentation de l'état dans Compose

Modifier l'apparence

- `TextField(`
- `// ...`
- `keyboardOptions = KeyboardOptions(),`
- `)`
- **Android permet de configurer le clavier à l'écran afin de permettre la saisie de chiffres, d'adresses e-mail, d'URL et de mots de passe, pour ne citer que quelques exemples.**
- Pour en savoir plus sur les autres types de claviers, consultez [KeyboardType](#).

Présentation de l'état dans Compose

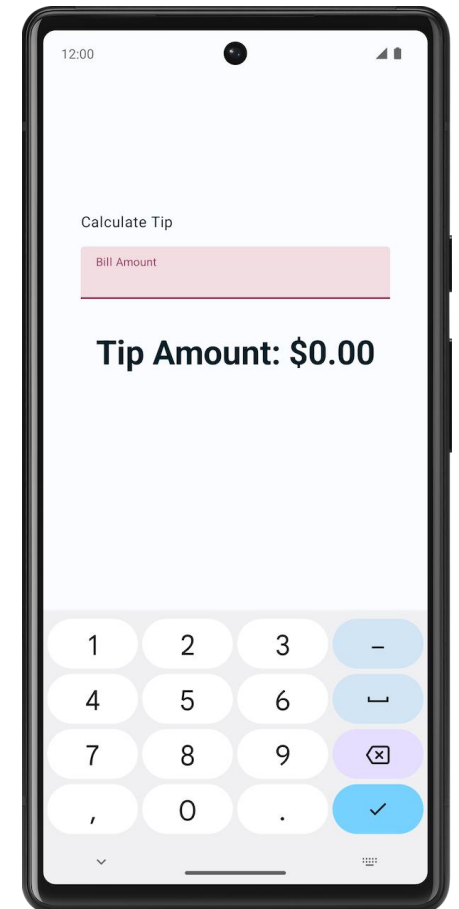
Modifier l'apparence

- Définissez le clavier numérique comme type de clavier pour saisir des chiffres.
- Transmettez à la fonction `KeyboardOptions` un paramètre `keyboardType` défini sur `KeyboardType.Number` :
 - `import androidx.compose.ui.text.input.KeyboardType`
 - `TextField(`
 - `// ...`
 - `keyboardOptions = KeyboardOptions(keyboardType =`
`KeyboardType.Number),`
 - `)`

Présentation de l'état dans Compose

Modifier l'apparence

- Exécutez l'application.
- Vous pouvez voir les modifications apportées au clavier dans cette capture d'écran :



Présentation de l'état dans Compose

Afficher le montant du pourboire

- Dans cette section, vous implémenterez la fonctionnalité principale de l'application, qui consiste à calculer et à afficher le montant du pourboire.
- Dans le fichier MainActivity.kt, une fonction private (privée) calculateTip() vous est fournie dans le code de démarrage.
- Vous l'utiliserez pour calculer le montant du pourboire comme suit :
 - `private fun calculateTip(amount: Double, tipPercent: Double = 15.0): String {`
 - `val tip = tipPercent / 100 * amount`
 - `return NumberFormat.getCurrencyInstance().format(tip)`
 - `}`

Présentation de l'état dans Compose

Afficher le montant du pourboire

- Dans la méthode ci-dessus, **vous utilisez NumberFormat pour appliquer un format monétaire au pourboire.**
- Votre application peut maintenant calculer le pourboire, mais vous devez encore le mettre en forme et l'afficher avec la classe adaptée.
- **Utiliser la fonction calculateTip()**
- Le texte saisi par l'utilisateur dans le composable du champ de texte est renvoyé à la fonction de rappel onValueChange en tant que String, même si l'utilisateur a saisi un nombre.
- Pour résoudre ce problème, vous devez convertir la valeur amountInput, qui correspond au montant saisi par l'utilisateur.

Présentation de l'état dans Compose

Afficher le montant du pourboire

- Dans la fonction composable `EditNumberField()`, créez une variable appelée `amount` après la définition de `amountInput`.
- Appelez la fonction `toDoubleOrNull` sur la variable `amountInput` pour la convertir de `String` en `Double` :
 - `val amount = amountInput.toDoubleOrNull()`
- La fonction `toDoubleOrNull()` est une fonction Kotlin prédéfinie qui analyse une chaîne en tant que nombre `Double` et renvoie le résultat, ou `null` si la chaîne ne correspond pas à une représentation valide d'un nombre.

Présentation de l'état dans Compose

Afficher le montant du pourboire

- À la fin de l'instruction, ajoutez un opérateur Elvis `?:` qui renverra une valeur 0.0 lorsque la valeur de `amountInput` sera nulle :
 - `val amount = amountInput.toDoubleOrNull() ?: 0.0`
- Remarque : L'opérateur Elvis `?:` renvoie l'expression qui le précède si la valeur n'est pas null et l'expression qui le suit lorsque la valeur est null.
- Après la variable `amount`, créez une autre variable `val` appelée `tip`.
- Initialisez-la avec l'élément `calculateTip()`, en transmettant le paramètre `amount`.
 - `val tip = calculateTip(amount)`

Présentation de l'état dans Compose

Afficher le montant du pourboire

- La fonction `EditNumberField()` devrait se présenter comme suit :

- `@Composable`
- `fun EditNumberField(modifier: Modifier = Modifier) {`
- `var amountInput by remember { mutableStateOf("") }`
- `val amount = amountInput.toDoubleOrNull() ?: 0.0`
- `val tip = calculateTip(amount)`
- `TextField(`
- `value = amountInput,`
- `onValueChange = { amountInput = it },`
- `label = { Text(stringResource(R.string.bill_amount)) },`
- `modifier = Modifier.fillMaxWidth(),`

Présentation de l'état dans Compose

Afficher le montant du pourboire

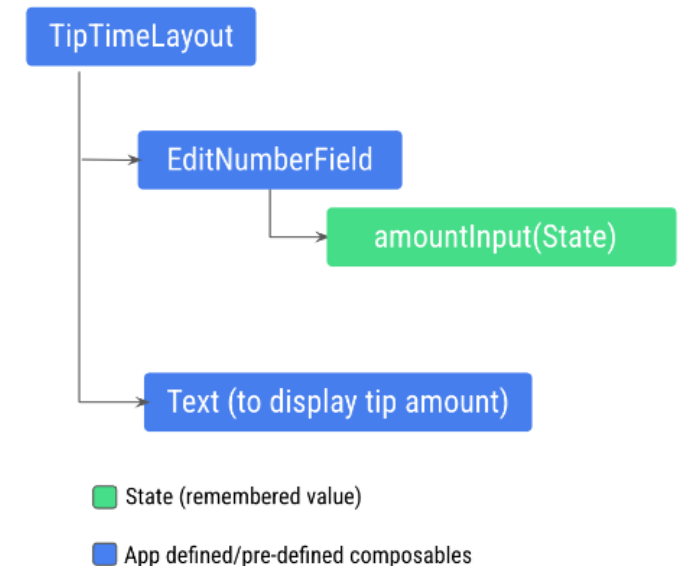
- `singleLine = true,`
- `keyboardOptions = KeyboardOptions(keyboardType =`
- `KeyboardType.Number)`
- `)`
- `}`

- **Afficher le montant calculé du pourboire**
- Vous avez écrit la fonction permettant de calculer le montant du pourboire.
- L'étape suivante consiste à afficher le montant calculé :
 - Dans la fonction `TipTimeLayout()`, à la fin du bloc `Column()`, un composable de texte affiche `$0.00`.

Présentation de l'état dans Compose

Afficher le montant du pourboire

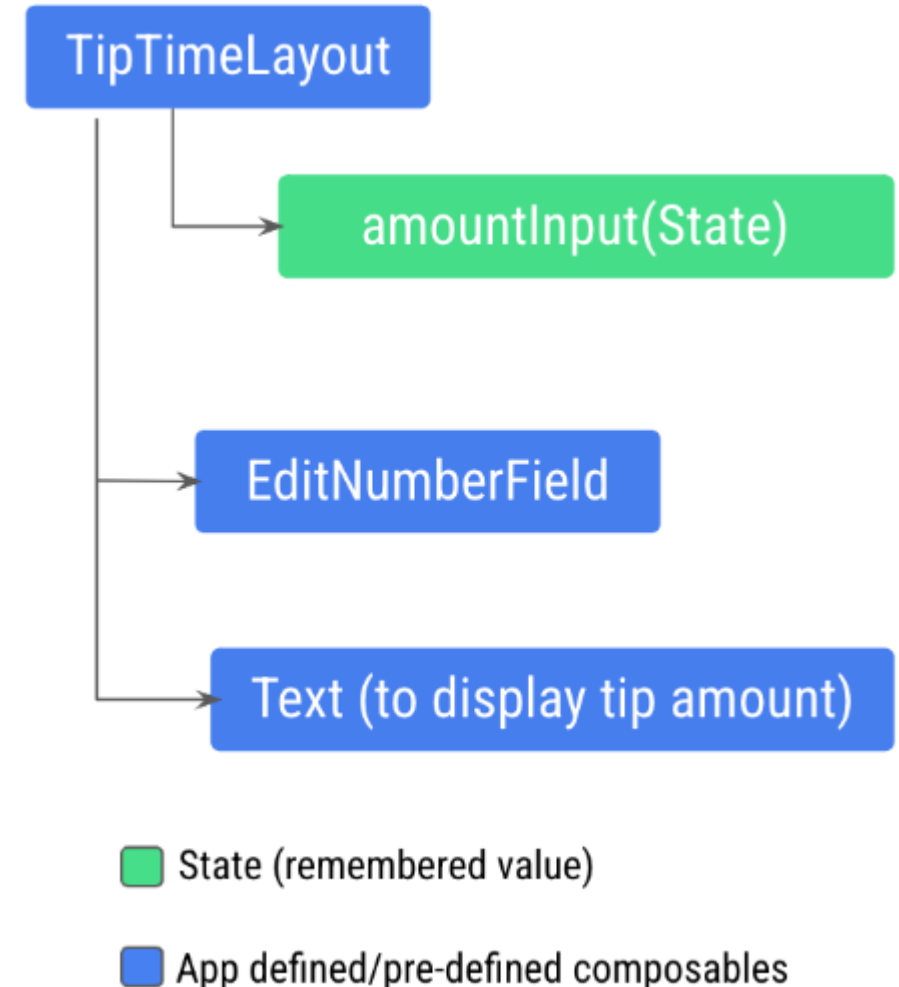
- Vous devez modifier cette valeur pour qu'elle corresponde au montant calculé du pourboire.
- Vous devez accéder à la variable `amountInput` dans la fonction `TipTimeLayout()` pour calculer et afficher le montant du pourboire, mais la variable `amountInput` correspond à l'état du champ de texte défini dans la fonction composable `EditNumberField()`. Vous ne pouvez pas encore l'appeler depuis la fonction `TipTimeLayout()`. Cette image illustre la structure du code :



Présentation de l'état dans Compose

Afficher le montant du pourboire

- Cette structure ne vous permet pas d'afficher le montant du pourboire dans le nouveau composable Text, car le composable Text doit accéder à la variable amount calculée à partir de la variable amountInput.
- Vous devez exposer la variable amount à la fonction TipTimeLayout().
- Cette image illustre la structure de code souhaitée, qui rend le composable EditNumberField() sans état :



Présentation de l'état dans Compose

Afficher le montant du pourboire

- Ce système est appelé **hissage d'état**.
- Dans la section suivante, vous allez hisser l'état d'un composable afin de le rendre sans état.
- Remarque : **Un composable sans état est un composable qui ne stocke pas son propre état. Il affiche n'importe quel état sous forme d'arguments d'entrée.**

Présentation de l'état dans Compose

Hisser un état

- Dans cette section, vous allez apprendre à choisir où définir l'état de sorte à pouvoir réutiliser et partager vos composables.
- Dans une fonction modulable, vous pouvez définir des variables qui contiennent un état à afficher dans l'interface utilisateur.
- Par exemple, vous avez défini la variable `amountInput` comme état dans le composable `EditNumberField()`.
- Lorsque l'application devient plus complexe et que d'autres composables ont besoin d'accéder à l'état au sein de ce composable `EditNumberField()`, vous devez envisager de **hisser ou extraire l'état** de la fonction modulable `EditNumberField()`.

Présentation de l'état dans Compose

Hisser un état

- **Comprendre les composables avec ou sans état**
- Vous devez hisser l'état dans les cas suivants :
 - Vous devez partager l'état avec plusieurs fonctions modulables.
 - Vous devez créer un composable sans état qui pourra être réutilisé dans votre application.
- **Lorsque vous extrayez l'état d'une fonction composable, la fonction composable obtenue est appelée sans état.**
- Autrement dit, les fonctions composables peuvent être rendues sans état si l'on extrait leur état.
- Un composable sans état est un composable qui n'a pas d'état.

Présentation de l'état dans Compose

Hisser un état

- En d'autres termes, il ne peut détenir, définir, ni modifier aucun nouvel état.
- En revanche, un composable avec état possède un état qui peut changer au fil du temps.
- Remarque : Dans les applications réelles, il est parfois difficile qu'un composable soit entièrement sans état, en fonction des responsabilités qui l'accompagnent. **Vous devez concevoir les composables de sorte qu'ils possèdent le moins d'états possible et que l'état soit hissé en cas de besoin, en les exposant dans l'API du composable.**

Présentation de l'état dans Compose

Hisser un état

- **Le hissage d'état est un modèle qui consiste à faire remonter un état vers son appelant pour obtenir un composant sans état.**
- Lorsqu'il est appliqué à des composables, cela implique souvent d'introduire deux paramètres dans le composable :
 - Un paramètre `value: T`, qui correspond à la valeur actuelle à afficher.
 - Un paramètre `onValueChange: (T) -> Unit` lambda de rappel, qui est déclenché lorsque la valeur change afin que l'état puisse être mis à jour ailleurs, par exemple lorsqu'un utilisateur saisit du texte dans la zone de texte.
- **Hissez l'état dans la fonction `EditNumberField()` :**
 - Mettez à jour la définition de la fonction `EditNumberField()` pour hisser l'état en ajoutant les paramètres `value` et `onValueChange` :

Présentation de l'état dans Compose

Hisser un état

- `@Composable`
- `fun EditNumberField(`
- `value: String,`
- `onValueChange: (String) -> Unit,`
- `modifier: Modifier = Modifier`
- `) {`
- `//...`
- Le paramètre `value` est de type `String`, et le paramètre `onValueChange` est de type `(String) -> Unit`.
- Il s'agit donc d'une fonction qui accepte une valeur `String` et n'a aucune valeur renvoyée.

Présentation de l'état dans Compose

Hisser un état

- Le paramètre `onValueChange` est utilisé comme rappel `onValueChange` transmis au composable `TextField`.
- Remarque : **une bonne pratique consiste à ajouter un paramètre `Modifier` par défaut à toutes les fonctions composables, ce qui améliore le potentiel de réutilisation. Vous devez l'ajouter comme premier paramètre facultatif, juste après tous les paramètres obligatoires.**
- Dans la fonction `EditNumberField()`, mettez à jour la fonction composable `TextField()` pour qu'elle utilise les paramètres transmis :

Présentation de l'état dans Compose

Hisser un état

- ```
TextField(
 value = value,
 onChange = onChange,
 // Rest of the code
)
```
- Hissez l'état, déplacez l'état mémorisé de la fonction `EditNumberField()` vers la fonction `TipTimeLayout()` :
  - ```
@Composable  
fun TipTimeLayout() {  
    var amountInput by remember { mutableStateOf("") }  
    val amount = amountInput.toDoubleOrNull() ?: 0.0
```

Présentation de l'état dans Compose

Hisser un état

- ```
val tip = calculateTip(amount)
```
  - ```
Column()
```
 - ```
//...
```
  - ```
) {
```
 - ```
//...
```
  - ```
}
```
 - ```
}
```
- Vous avez hissé l'état vers `TipTimeLayout()`.
  - Transmettez-le maintenant à `EditNumberField()`.



# Présentation de l'état dans Compose

## Hisser un état

- Dans la fonction `TipTimeLayout()`, mettez à jour l'appel de fonction `EditNumberField()` pour utiliser l'état hissé :
  - `EditNumberField(`
  - `value = amountInput,`
  - `onValueChange = { amountInput = it },`
  - `modifier = Modifier`
  - `.padding(bottom = 32.dp)`
  - `.fillMaxWidth()`
  - `)`
- De ce fait, `EditNumberField` devient sans état.

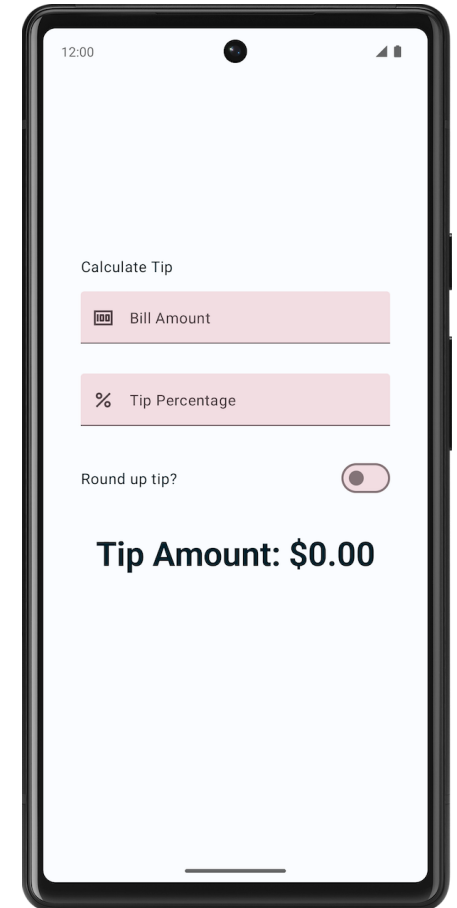
# Présentation de l'état dans Compose

## Hisser un état

- Vous avez hissé l'état de l'interface utilisateur vers son ancêtre, `TipTimeLayout()`.
- `TipTimeLayout()` est désormais le propriétaire de l'état `amountInput`.
- **Modifier le code pour afficher le montant du pourboire.**

# Calculer un pourboire personnalisé

- Dans cet atelier de programmation, vous utiliserez le code de l'atelier de programmation précédent (Présentation de l'état dans Compose) afin de créer une calculatrice de pourboire interactive.
- Celle-ci permettra de calculer et d'arrondir automatiquement le montant d'un pourboire lorsque vous saisissez le montant de la note et le pourcentage de pourboire.
- Cette image représente l'application finale :



# Calculer un pourboire personnalisé

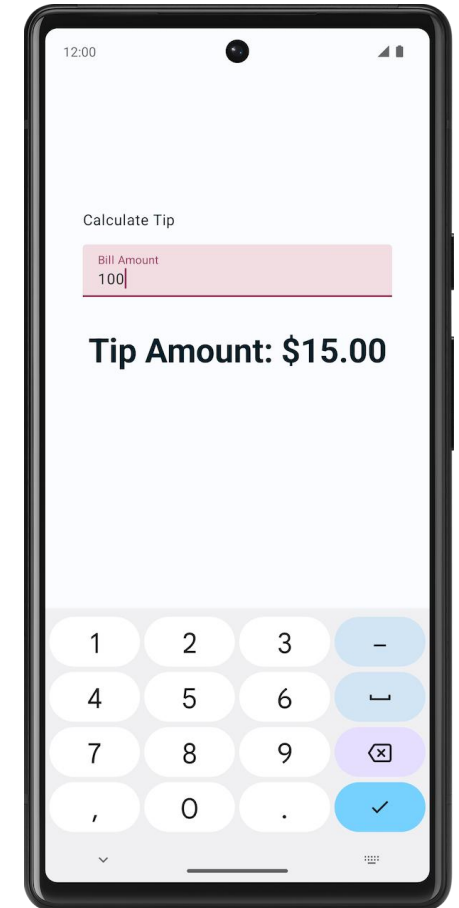
## Présentation de l'application de démarrage

- Cet atelier de programmation commence avec l'application Tip Time issue de l'atelier de programmation précédent Présentation de l'état dans Compose.
- Il fournit l'interface utilisateur nécessaire pour calculer un pourboire avec un pourcentage fixe.
- La zone de texte Bill amount (Montant de la facture) permet à l'utilisateur de saisir le coût du service.
- L'application calcule et affiche le montant du pourboire dans un composable Text.

# Calculer un pourboire personnalisé

## Présentation de l'application de démarrage

- **Exécuter l'application Tip Time**
- Ouvrez le projet Tip Time dans Android Studio, puis exécutez l'application sur un émulateur ou un appareil.
- Saisissez un montant de facture.
- L'application calcule et affiche automatiquement le montant du pourboire.



# Calculer un pourboire personnalisé

## Présentation de l'application de démarrage

- Dans la configuration actuelle, le pourcentage de pourboire est codé en dur et se monte à 15 %.
- Dans cet atelier de programmation, vous ajouterez à cette fonctionnalité un champ de texte permettant à l'application de calculer un pourcentage de pourboire personnalisé et d'arrondir son montant.
- **Ajouter les ressources de chaîne nécessaires**
- Dans l'onglet Projet, cliquez sur res > values > string.xml.
- Entre les balises <resources> du fichier strings.xml, ajoutez les ressources de chaîne suivantes :

# Calculer un pourboire personnalisé

## Présentation de l'application de démarrage

- `<string name="how_was_the_service">Tip Percentage</string>`
- `<string name="round_up_tip">Round up tip?</string>`
- Le fichier strings.xml doit ressembler à cet extrait de code, qui inclut les chaînes de l'atelier de programmation précédent :
- strings.xml
  - `<resources>`
  - `<string name="app_name">Tip Time</string>`
  - `<string name="calculate_tip">Calculate Tip</string>`
  - `<string name="bill_amount">Bill Amount</string>`
  - `<string name="how_was_the_service">Tip Percentage</string>`

# Calculer un pourboire personnalisé

Présentation de l'application de démarrage

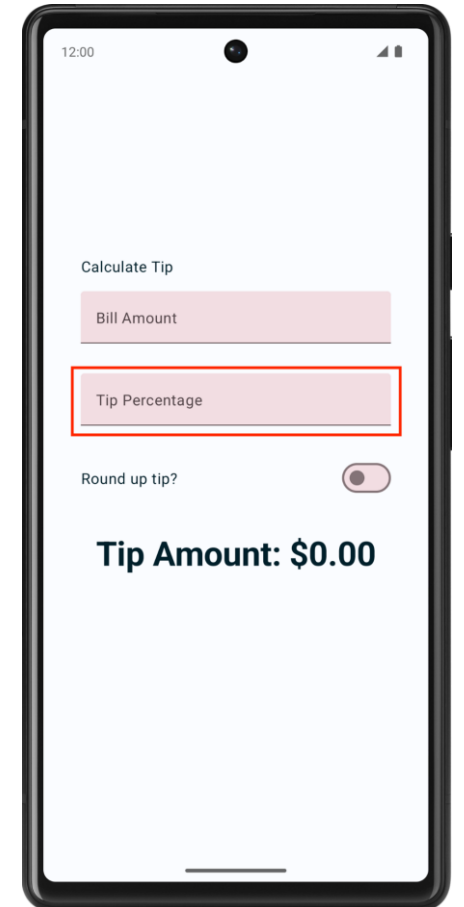
- `<string name="round_up_tip">Round up tip?</string>`
- `<string name="tip_amount">Tip Amount: %s</string>`
- `</resources>`



# Calculer un pourboire personnalisé

Ajouter un champ de texte pour le pourcentage de pourboire

- Le pourboire peut être plus ou moins élevé en fonction de la qualité du service fourni et d'autres raisons.
- C'est pourquoi l'application doit permettre à l'utilisateur de calculer un pourboire personnalisé.
- Dans cette section, vous ajouterez un champ de texte dans lequel l'utilisateur pourra saisir un pourcentage de pourboire personnalisé, comme illustré sur l'image suivante :



# Calculer un pourboire personnalisé

Ajouter un champ de texte pour le pourcentage de pourboire

- Vous disposez déjà d'un composable de champ de texte Montant de la facture dans votre application, qui est la fonction modulable `EditNumberField()` sans état.
- Dans l'atelier de programmation précédent, vous avez hissé l'état `amountInput` du composable `EditNumberField()` vers le composable `TipTimeLayout()`.
- C'est pour cela que le composable `EditNumberField()` est devenu sans état.
- Pour ajouter un champ de texte, vous pouvez réutiliser le même composable `EditNumberField()`, mais avec un libellé différent.

# Calculer un pourboire personnalisé

Ajouter un champ de texte pour le pourcentage de pourboire

- Pour effectuer cette modification, vous devez transmettre le libellé en tant que paramètre plutôt que de le coder en dur dans la fonction modulaire `EditNumberField()`.
- Rendez la fonction modulaire `EditNumberField()` réutilisable :
  - Dans le fichier `MainActivity.kt` dans les paramètres de la fonction modulaire `EditNumberField()`, ajoutez une ressource de chaîne label de type `Int` :
  - `@Composable`
  - `fun EditNumberField(`
  - `label: Int,`
  - `value: String,`
  - `onValueChanged: (String) -> Unit,`

# Calculer un pourboire personnalisé

Ajouter un champ de texte pour le pourcentage de pourboire

- `modifier: Modifier = Modifier`
- `)`
- Dans le corps de la fonction, remplacez l'ID de ressource de la chaîne codée en dur par le paramètre `label` :
  - `@Composable`
  - `fun EditNumberField(`
  - `//...`
  - `) {`
  - `TextField(`
  - `//...`
  - `label = { Text(stringResource(label)) },`
  - `//...`
  - `)`
  - `}`

# Calculer un pourboire personnalisé

Ajouter un champ de texte pour le pourcentage de pourboire

- Pour indiquer que le paramètre label doit être une référence de ressource de chaîne, annotez le paramètre de fonction avec la mention **@StringRes** :

- `@Composable`
- `fun EditNumberField(`
- `@StringRes label: Int,`
- `value: String,`
- `onValueChanged: (String) -> Unit,`
- `modifier: Modifier = Modifier`
- `)`

# Calculer un pourboire personnalisé

Ajouter un champ de texte pour le pourcentage de pourboire

- Remarque : **L'annotation `@StringRes` est un moyen sûr d'utiliser des ressources de chaîne.** Elle indique que l'entier à transmettre est une **ressource de chaîne provenant du fichier `values/strings.xml`.** Ces annotations sont utiles aux développeurs qui travaillent sur votre code et aux outils d'inspection de code tels que lint dans Android Studio. Vous vous familiariserez avec lint dans un atelier de programmation ultérieur.
- Dans l'appel `EditNumberField()` de la fonction composable `TipTimeLayout()`, définissez le paramètre `label` sur la ressource de chaîne `R.string.bill_amount` :

# Calculer un pourboire personnalisé

Ajouter un champ de texte pour le pourcentage de pourboire

- `EditNumberField(`
- `label = R.string.bill_amount,`
- `value = amountInput,`
- `onValueChanged = { amountInput = it },`
- `modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()`
- `)`
- Dans la fonction composable `TipTimeLayout()` après l'appel de `EditNumberField()`, ajoutez un autre champ de texte pour le pourcentage de pourboire personnalisé.
- Appelez la fonction modulable `EditNumberField()` avec les paramètres suivants :

# Calculer un pourboire personnalisé

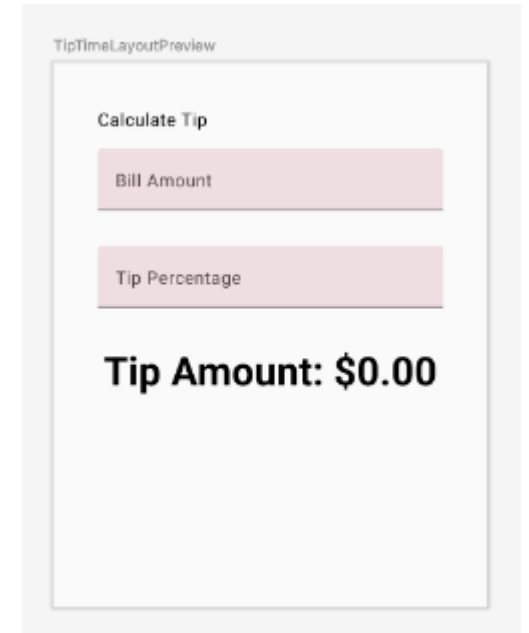
Ajouter un champ de texte pour le pourcentage de pourboire

- ```
EditNumberField(  
    label = R.string.how_was_the_service,  
    value = "",  
    onChange = { },  
    modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()  
)
```
- Une zone de texte supplémentaire est ajoutée pour le pourcentage de pourboire personnalisé.
- L'aperçu de l'application affiche désormais un champ de texte Tip Percentage (Pourcentage de pourboire), comme illustré sur cette image :

Calculer un pourboire personnalisé

Ajouter un champ de texte pour le pourcentage de pourboire

- En haut de la fonction composable `TipTimeLayout()`, ajoutez une propriété `var` appelée `tipInput` pour la variable d'état du champ de texte ajouté.
- Utilisez `mutableStateOf("")` pour initialiser la variable et délimitez l'appel avec la fonction `remember` :
 - **`var tipInput by remember { mutableStateOf("") }`**
- Dans le nouvel appel de fonction `EditNumberField()`, définissez le paramètre `value` sur la variable `tipInput`, puis mettez à jour la variable `tipInput` dans l'expression `lambda onValueChanged` :



Calculer un pourboire personnalisé

Ajouter un champ de texte pour le pourcentage de pourboire

- `EditNumberField(`
- `label = R.string.how_was_the_service,`
- `value = tiplInput,`
- `onValueChange = { tiplInput = it },`
- `modifier = Modifier.padding(bottom = 32.dp).fillMaxWidth()`
- `)`
- Dans la fonction `TipTimeLayout()` après la définition de la variable `tiplInput` définissez une variable `val` nommée `tipPercent` qui convertit la variable `tiplInput` en un type `Double`.
- Utilisez un opérateur Elvis et renvoyez 0, si la valeur est null.
- Cette valeur peut être null si le champ de texte est vide.

Calculer un pourboire personnalisé

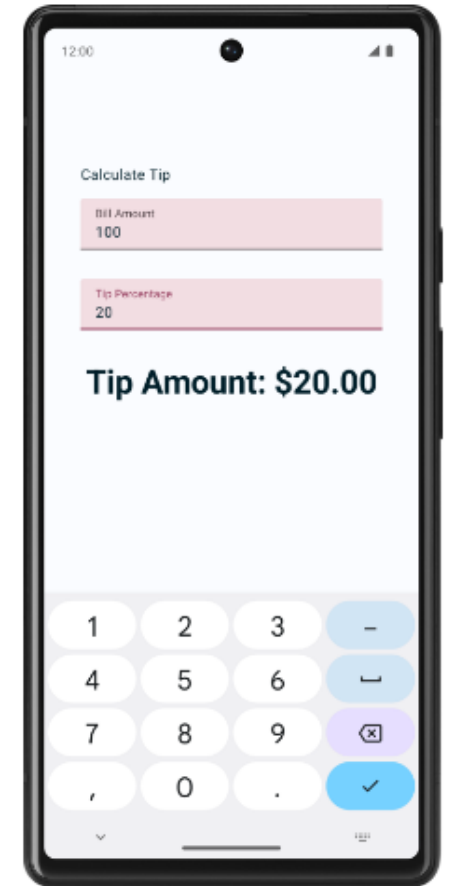
Ajouter un champ de texte pour le pourcentage de pourboire

- `val tipPercent = tipInput.toDoubleOrNull() ?: 0.0`
- Dans la fonction `TipTimeLayout()`, mettez à jour l'appel de fonction `calculateTip()` et transmettez la variable `tipPercent` en tant que deuxième paramètre :
 - `val tip = calculateTip(amount, tipPercent)`

Calculer un pourboire personnalisé

Ajouter un champ de texte pour le pourcentage de pourboire

- Exécutez l'application sur un émulateur ou un appareil, puis saisissez le montant de la facture et le pourcentage de pourboire.
- L'application calcule-t-elle correctement le montant du pourboire ?



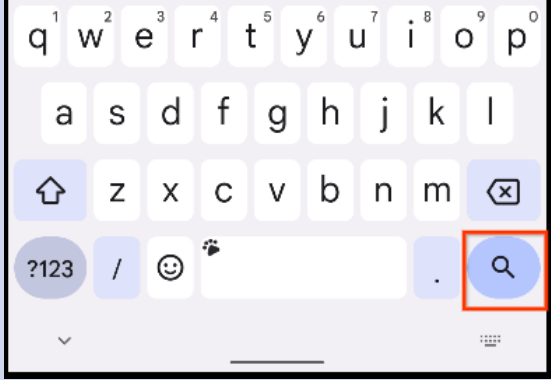
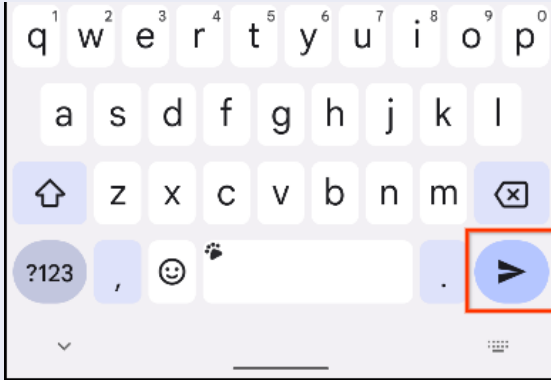
Calculer un pourboire personnalisé

Définir un bouton d'action

- Dans l'atelier de programmation précédent, vous avez appris à utiliser la classe `KeyboardOptions` pour définir le type de clavier.
- Dans cette section, vous allez découvrir comment définir le bouton d'action du clavier avec la même classe, `KeyboardOptions`.
- **Un bouton d'action est un bouton situé à l'extrémité du clavier.**
- Voici quelques exemples dans ce tableau :

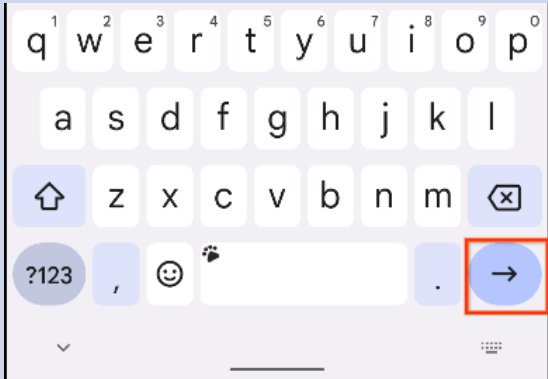
Calculer un pourboire personnalisé

Définir un bouton d'action

Propriété	Bouton d'action du clavier
ImeAction.Search Option possible lorsque l'utilisateur souhaite effectuer une recherche.	 A virtual keyboard layout is shown. The bottom row of keys includes a "?123" key, a "/" key, a smiley face icon, a paw print icon, a spacebar, a period key, and a blue circular key with a white magnifying glass icon. This magnifying glass key is highlighted with a red rectangular box.
ImeAction.Send Option possible lorsque l'utilisateur souhaite envoyer le texte dans le champ de saisie.	 A virtual keyboard layout is shown, identical to the one above. The bottom row of keys includes a "?123" key, a "," key, a smiley face icon, a paw print icon, a spacebar, a period key, and a blue circular key with a white right-pointing arrow icon. This arrow key is highlighted with a red rectangular box.

Calculer un pourboire personnalisé

Définir un bouton d'action

Propriété	Bouton d'action du clavier
ImeAction.Go Transforme la touche "Entrée" en bouton "Go" ou "Aller".	 A virtual keyboard interface is shown. The keyboard has four rows of keys. The first row contains letters q, w, e, r, t, y, u, i, o, p with numbers 1 through 0 above them. The second row contains a, s, d, f, g, h, j, k, l. The third row contains a home key (house icon), z, x, c, v, b, n, m, and a delete key (X in a box). The fourth row contains a ?123 key, a comma key, a smiley face key, a paw print key, a search key (magnifying glass), a period key, and a right arrow key. The right arrow key is highlighted with a red square border.

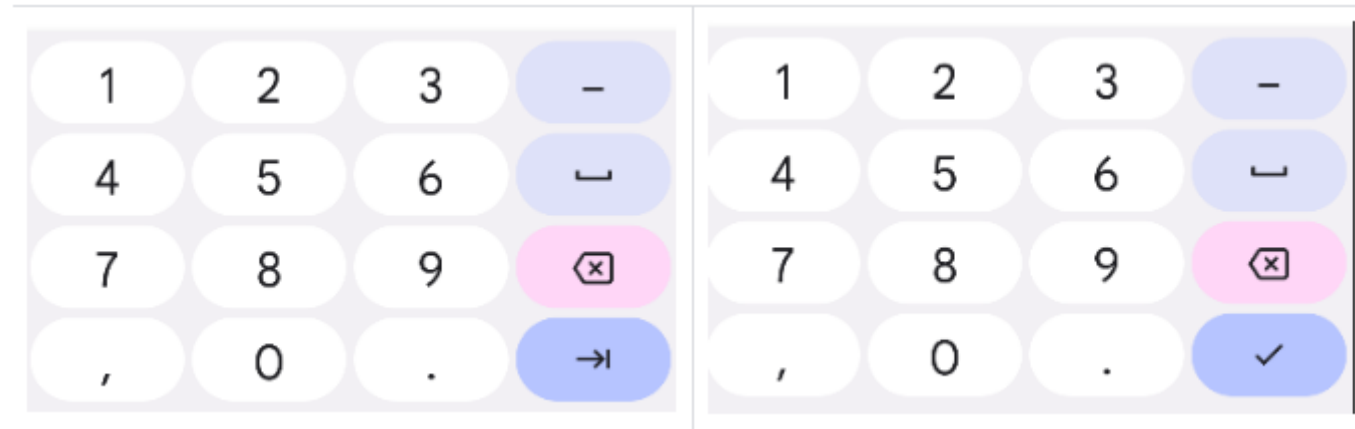
Calculer un pourboire personnalisé

Définir un bouton d'action

- Au cours de cette tâche, vous définirez deux boutons d'action différents pour les zones de texte :
 - Un bouton d'action Next (Suivant) pour la zone de texte Bill Amount (Montant de la facture), qui indique que l'utilisateur a terminé la saisie et souhaite passer à la zone de texte suivante.
 - Un bouton d'action Done (Terminé) pour la zone de texte Tip Percentage (Pourcentage de pourboire), qui indique que l'utilisateur a terminé de saisir une entrée.
- Voici, sur les images suivantes, des exemples de claviers comportant ces boutons d'action :

Calculer un pourboire personnalisé

Définir un bouton d'action



- Ajouter des options de clavier :
 - Dans l'appel `TextField()` de la fonction `EditNumberField()`, transmettez au constructeur `KeyboardOptions` un argument nommé `imeAction` défini sur une valeur `ImeAction.Next`. Utilisez la fonction `KeyboardOptions.Default.copy()` pour vous assurer d'utiliser les autres options par défaut.

Calculer un pourboire personnalisé

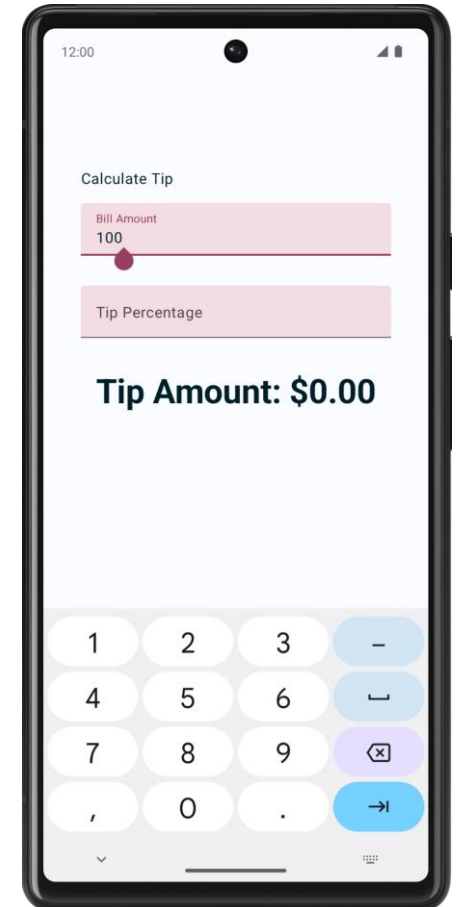
Définir un bouton d'action

- `import androidx.compose.ui.text.input.ImeAction`
- `@Composable`
- `fun EditNumberField(`
- `//...`
- `) {`
- `TextField(`
- `//...`
- `keyboardOptions = KeyboardOptions.Default.copy(`
- `keyboardType = KeyboardType.Number,`
- `imeAction = ImeAction.Next`
- `),`

Calculer un pourboire personnalisé

Définir un bouton d'action

- `)`
- `}`
- Exécutez l'application sur un émulateur ou un appareil.
- Le clavier affiche maintenant le bouton d'action Next (Suivant), comme illustré sur cette image :



Calculer un pourboire personnalisé

Définir un bouton d'action

- Notez que le même bouton d'action Next (Suivant) s'affiche sur le clavier lorsque le champ de texte Tip Percentage (Pourcentage de pourboire) est sélectionné.
- Toutefois, vous avez besoin de deux boutons d'action différents pour les champs de texte.
- Examinez la fonction `EditNumberField()`.
- Le paramètre `keyboardOptions` de la fonction `TextField()` est codé en dur.

Calculer un pourboire personnalisé

Définir un bouton d'action

- Pour créer différents boutons d'action pour les champs de texte, vous devez transmettre l'objet `KeyboardOptions` en tant qu'argument, ce que vous ferez à l'étape suivante.
- Dans la définition de la fonction `EditNumberField()`, ajoutez un paramètre `keyboardOptions` de type `KeyboardOptions`. Dans le corps de la fonction, attribuez-le au paramètre nommé `keyboardOptions` de la fonction `TextField()` :
 - `@Composable`
 - `fun EditNumberField(`
 - `@StringRes label: Int,`
 - `keyboardOptions: KeyboardOptions,`

Calculer un pourboire personnalisé

Définir un bouton d'action

- `// ...`
 - `)`
 - `TextField(`
 - `//...`
 - **`keyboardOptions = keyboardOptions`**
 - `)`
 - `}`
- Dans la fonction `TipTimeLayout()`, modifiez le premier appel de fonction `EditNumberField()`, puis transmettez le paramètre nommé `keyboardOptions` pour le champ de texte `Bill Amount` (Montant de la facture) :

Calculer un pourboire personnalisé

Définir un bouton d'action

- `EditNumberField(`
 - `label = R.string.bill_amount,`
 - `keyboardOptions = KeyboardOptions.Default.copy(`
 - `keyboardType = TextInputType.Number,`
 - `imeAction = ImeAction.Next`
 - `),`
 - `// ...`
 - `)`
- Dans le second appel de fonction `EditNumberField()`, remplacez la valeur `imeAction` du champ de texte `Tip Percentage (Pourcentage de pourboire)` par `ImeAction.Done`. La fonction devrait se présenter comme suit :

Calculer un pourboire personnalisé

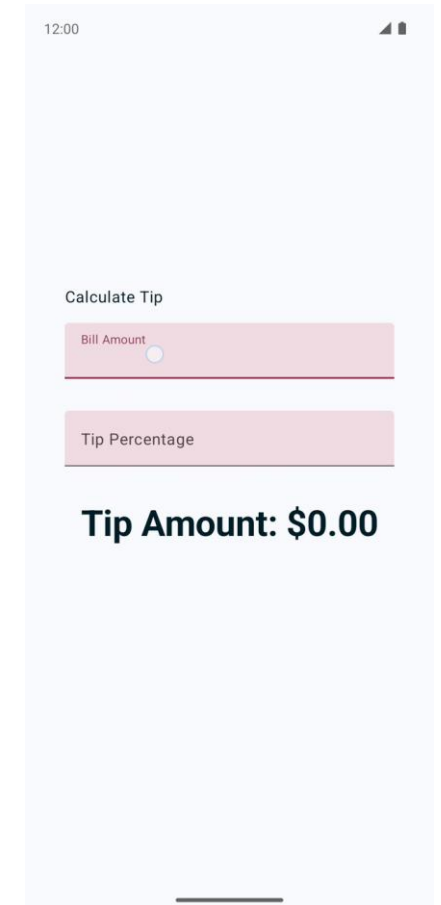
Définir un bouton d'action

- `EditNumberField(`
- `label = R.string.how_was_the_service,`
- `keyboardOptions = KeyboardOptions.Default.copy(`
- `keyboardType = TextInputType.Number,`
- `imeAction = ImeAction.Done`
- `),`
- `// ...`
- `)`
- Exécutez l'application. Les boutons Suivant et Terminé s'affichent, comme illustré sur les images suivantes :

Calculer un pourboire personnalisé

Définir un bouton d'action

- Saisissez le montant d'une facture, cliquez sur le bouton d'action Next (Suivant), saisissez un pourcentage de pourboire, puis cliquez sur le bouton d'action Done (Terminé).
- Le clavier se ferme.

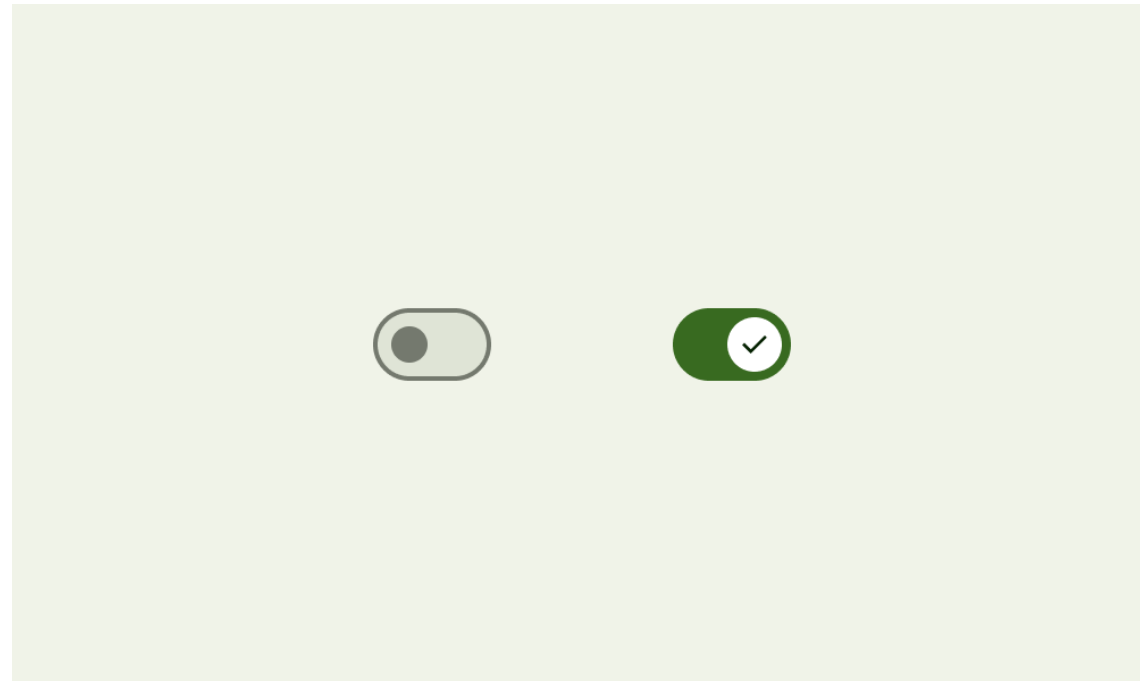


The screenshot shows a mobile application interface for calculating a tip. At the top, the status bar displays the time 12:00 and signal strength. The app title "Calculate Tip" is centered. Below the title, there are two input fields: "Bill Amount" and "Tip Percentage". The "Bill Amount" field has a small blue circle next to it, indicating it is the active input field. Below these fields, the text "Tip Amount: \$0.00" is displayed in a bold font. The interface is clean and minimalist, with a light blue background and pinkish-red input fields.

Calculer un pourboire personnalisé

Ajouter un bouton bascule

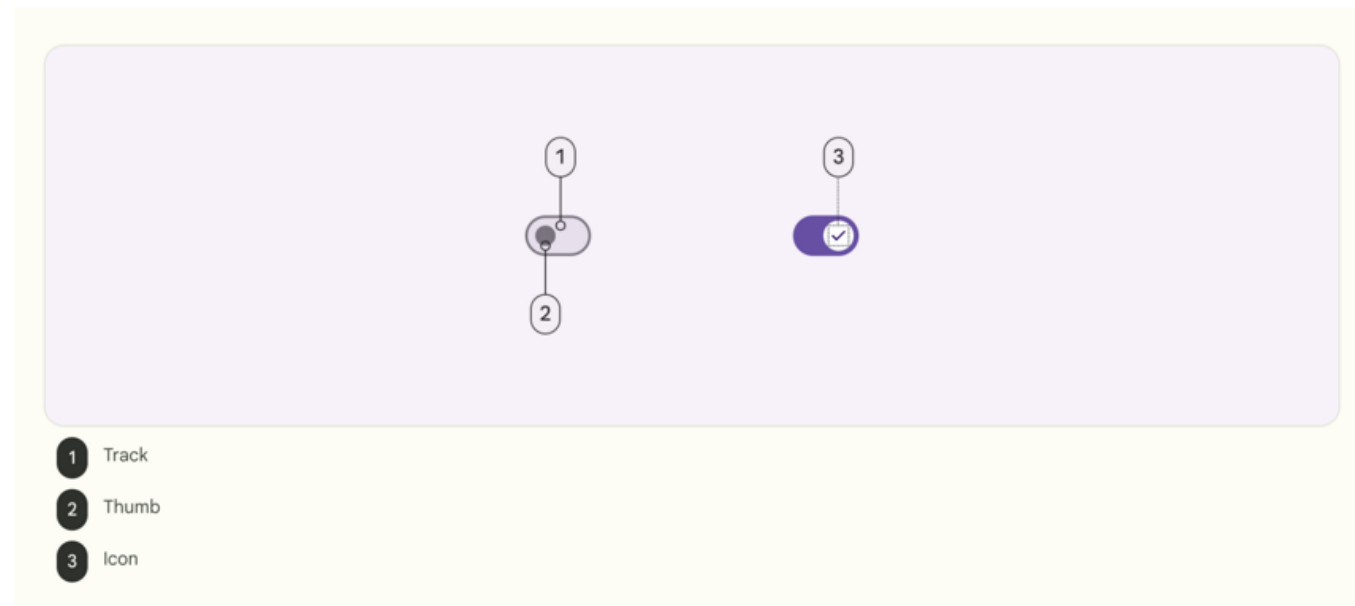
- Un bouton bascule permet d'activer ou de désactiver l'état d'un unique élément.



Calculer un pourboire personnalisé

Ajouter un bouton bascule

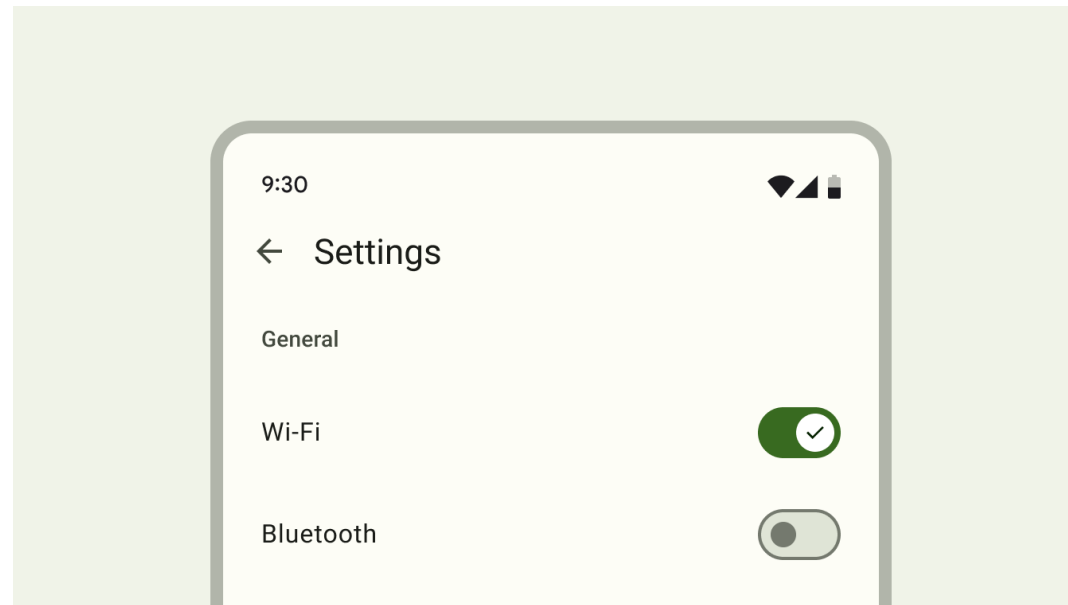
- Un bouton bascule à deux états permet à l'utilisateur de choisir entre deux options.
- Un bouton bascule se compose d'un conteneur, d'un curseur et d'une icône facultative, comme illustré sur les images suivantes :



Calculer un pourboire personnalisé

Ajouter un bouton bascule

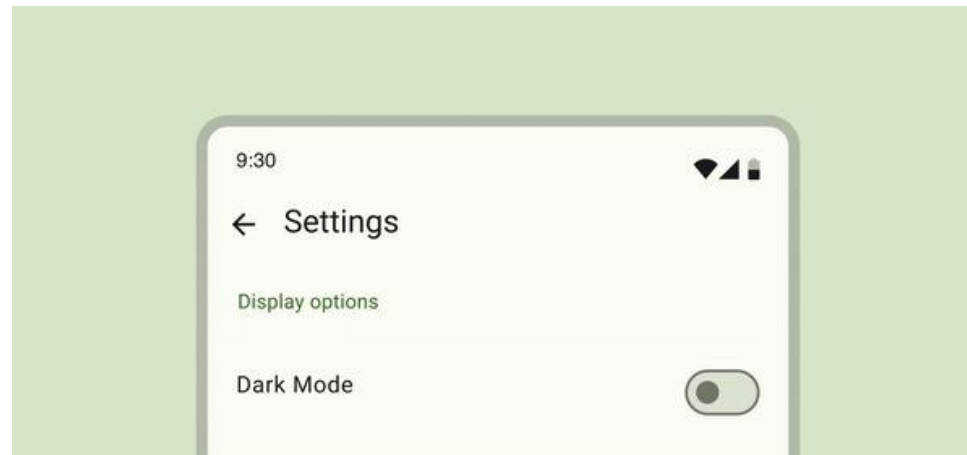
- Le bouton bascule est une commande de sélection qui vous permet d'indiquer un choix ou de déclarer une préférence, telles que des paramètres, comme illustré dans cette image :



Calculer un pourboire personnalisé

Ajouter un bouton bascule

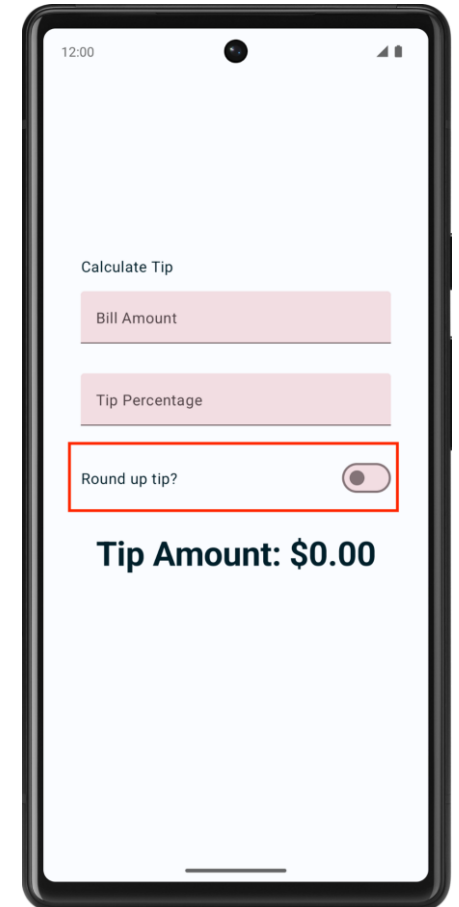
- L'utilisateur peut soit faire glisser le curseur d'avant en arrière pour définir l'option de son choix, soit simplement appuyer sur le bouton pour l'activer.
- Vous trouverez un autre exemple d'activation et de désactivation dans l'image GIF ci-dessous, où le paramètre "Display options" (Options visuelles) est défini sur Dark mode (Mode sombre) :



Calculer un pourboire personnalisé

Ajouter un bouton bascule

- Pour en savoir plus, consultez la documentation sur [les boutons bascules](#).
- Vous pouvez utiliser le composable **Switch** pour que l'utilisateur ait la possibilité d'arrondir le pourboire au nombre entier le plus proche, comme illustré dans cette image :



Calculer un pourboire personnalisé

Ajouter un bouton bascule

- Ajoutez une ligne pour les composables Text et Switch :
 - Après la fonction EditNumberField(), ajoutez une fonction modulable RoundTheTipRow(), puis transmettez un élément Modifier par défaut comme des arguments semblables à la fonction EditNumberField() :
 - `@Composable`
 - `fun RoundTheTipRow(modifier: Modifier = Modifier) {`
 - `}`
- Implémentez la fonction RoundTheTipRow(), ajoutez un composable Row de mise en page avec l'élément modifier suivant afin de définir la largeur maximale à l'écran pour les éléments enfants.

Calculer un pourboire personnalisé

Ajouter un bouton bascule

- Centrez l'alignement et assurez-vous que la taille correspond à une taille de 48dp :
 - `Row(`
 - `modifier = modifier`
 - `.fillMaxWidth()`
 - `.size(48.dp),`
 - `verticalAlignment = Alignment.CenterVertically`
 - `) {`
 - `}`

Calculer un pourboire personnalisé

Ajouter un bouton bascule

- Exécutez la commande d'importation suivante :
 - `import androidx.compose.foundation.layout.Row`
 - `import androidx.compose.foundation.layout.size`
- Dans le bloc lambda du composable de mise en page Row, ajoutez un composable Text qui utilisera la ressource de chaîne `R.string.round_up_tip` pour afficher une chaîne Round up tip? :
 - `Text(text = stringResource(R.string.round_up_tip))`
- Après le composable Text, ajoutez un composable **Switch**, puis transmettez un paramètre nommé `checked` à `roundUp` et un paramètre `onCheckedChange` défini sur `onRoundUpChanged`.

Calculer un pourboire personnalisé

Ajouter un bouton bascule

- `Switch(`
- `checked = roundUp,`
- `onCheckedChange = onRoundUpChanged,`
- `)`
- Ce tableau contient des informations sur ces paramètres, qui sont les mêmes que ceux que vous avez définis pour la fonction `RoundTheTipRow()` :

Paramètre	Description
Checked	Indique si le bouton bascule est activé. Il s'agit de l'état du composable <code>Switch</code> .
onCheckedChange	Rappel à appeler en cas de clic sur le bouton bascule.

Calculer un pourboire personnalisé

Ajouter un bouton bascule

- Exécutez la commande d'importation suivante :
 - `import androidx.compose.material3.Switch`
- Dans la fonction `RoundTheTipRow()`, ajoutez un paramètre `roundUp` de type `Boolean` et une fonction lambda `onRoundUpChanged` qui accepte une valeur `Boolean` et ne renvoie rien :
 - `@Composable`
 - `fun RoundTheTipRow(`
 - `roundUp: Boolean,`
 - `onRoundUpChanged: (Boolean) -> Unit,`
 - `modifier: Modifier = Modifier`
 - `)`

Calculer un pourboire personnalisé

Ajouter un bouton bascule

- Cette action hisse l'état du bouton bascule.
- Dans le composable Switch, ajoutez cet élément modifier pour aligner le composable Switch avec la fin de l'écran :
 - `Switch(`
 - `modifier = modifier`
 - `.fillMaxWidth()`
 - `.wrapContentSize(Alignment.End),`
 - `//...`
 - `)`

Calculer un pourboire personnalisé

Ajouter un bouton bascule

- Exécutez la commande d'importation suivante :
 - `import androidx.compose.foundation.layout.wrapContentWidth`
- Dans la fonction `TipTimeLayout()`, ajoutez une variable `var` pour l'état du composable `Switch`.
- Créez une variable `var` nommée `roundUp` et définissez-la sur `mutableStateOf()`, avec `false` comme valeur initiale.
- Délimitez l'appel avec `remember { }`.
 - `fun TipTimeLayout() {`
 - `//...`
 - `var roundUp by remember { mutableStateOf(false) }`

Calculer un pourboire personnalisé

Ajouter un bouton bascule

- `//...`
- `Column(`
- `...`
- `) {`
- `//...`
- `}`
- `}`

- Il s'agit de la variable d'état du composable Switch.
- "false" est l'état par défaut.

Calculer un pourboire personnalisé

Ajouter un bouton bascule

- Dans le bloc Column de la fonction TipTimeLayout() après le champ de texte Tip Percentage (Pourcentage de pourboire), appelez la fonction RoundTheTipRow() avec les arguments suivants : paramètre nommé roundUp défini sur roundUp et paramètre nommé onRoundUpChanged défini sur un rappel lambda qui met à jour la valeur roundUp :

- @Composable

- fun TipTimeLayout() {
 - //...
 - Column(
 - ...

Calculer un pourboire personnalisé

Ajouter un bouton bascule

- `) {`
- `Text(`
- `...`
- `)`
- `Spacer(...)`
- `EditNumberField(`
- `...`
- `)`
- `EditNumberField(`
- `...`
- `)`

Calculer un pourboire personnalisé

Ajouter un bouton bascule

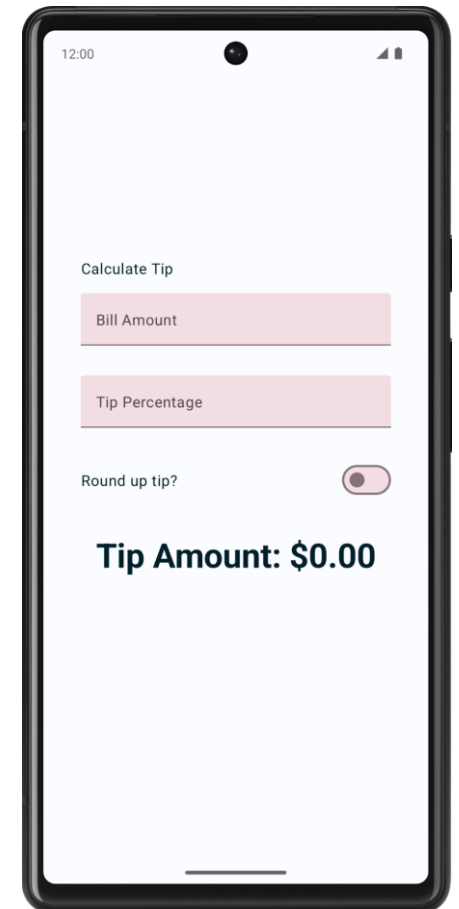
- `RoundTheTipRow(`
- `roundUp = roundUp,`
- `onRoundUpChanged = { roundUp = it },`
- `modifier = Modifier.padding(bottom = 32.dp)`
- `)`
- `Text(`
- `...`
- `)`
- `}`
- `}`

- La ligne Round up tip? (Arrondir le montant du pourboire ?) s'affiche.

Calculer un pourboire personnalisé

Ajouter un bouton bascule

- Exécutez l'application.
- Le bouton bascule Round up tip? (Arrondir le montant du pourboire ?) s'affiche.
- Saisissez le montant de la facture et le pourcentage de pourboire, puis cliquez sur le bouton Round up tip? (Arrondir le montant du pourboire ?).
- Le montant du pourboire n'est pas arrondi, car vous n'avez pas encore mis à jour la fonction `calculateTip()`, ce que vous ferez dans la section suivante.



Calculer un pourboire personnalisé

Ajouter un bouton bascule

- **Mettre à jour la fonction `calculateTip()` pour arrondir le pourboire**
- Modifiez la fonction `calculateTip()` pour qu'elle accepte une variable Boolean afin d'arrondir le pourboire à l'entier le plus proche :
 - Pour arrondir le pourboire, la fonction `calculateTip()` doit connaître l'état du bouton bascule, qui correspond à une valeur Boolean. Dans la fonction `calculateTip()`, ajoutez un paramètre `roundUp` de type Boolean :
 - `private fun calculateTip(`
 - `amount: Double,`
 - `tipPercent: Double = 15.0,`
 - `roundUp: Boolean`
 - `): String {`

Calculer un pourboire personnalisé

Ajouter un bouton bascule

- `//...`
- `}`
- Dans la fonction `calculateTip()`, avant l'instruction `return`, ajoutez une condition `if()` qui vérifiera la valeur `roundUp`.
- Si la valeur de l'élément `roundUp` est `true`, définissez une variable `tip` et ajoutez `kotlin.math.ceil()`, puis transmettez la variable `tip` comme argument :
 - `if (roundUp) {`
 - `tip = kotlin.math.ceil(tip)`
 - `}`

Calculer un pourboire personnalisé

Ajouter un bouton bascule

- Android Studio affiche une erreur.
- L'erreur `val cannot be reassigned` indique que vous devez remplacer la variable `tip` par une variable `var` :
 - `var tip = tipPercent / 100 * amount`
- Remarque : La fonction `kotlin.math.ceil(x)` arrondit la valeur donnée à l'entier supérieur. Par exemple, il arrondit la valeur 10,65 à 11. Cette fonction accepte les valeurs `Double` ou `Float`.
- La fonction `calculateTip()` terminée devrait ressembler à ceci :
 - `private fun calculateTip(amount: Double, tipPercent: Double = 15.0, roundUp: Boolean): String {`

Calculer un pourboire personnalisé

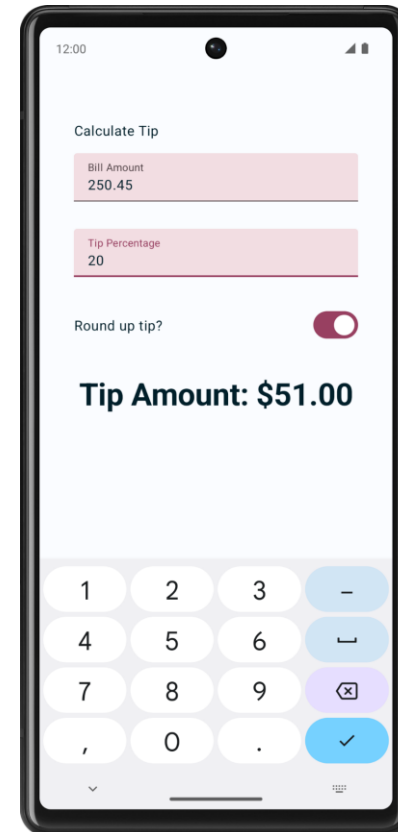
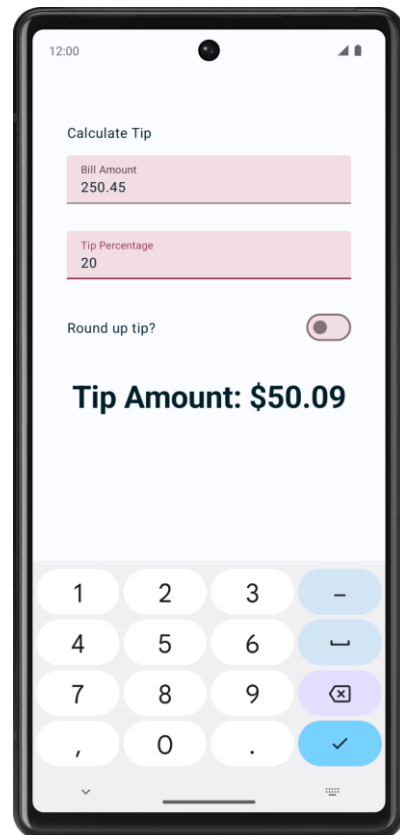
Ajouter un bouton bascule

- ```
var tip = tipPercent / 100 * amount
```
- ```
if (roundUp) {
```
- ```
 tip = kotlin.math.ceil(tip)
```
- ```
}
```
- ```
return NumberFormat.getCurrencyInstance().format(tip)
```
- ```
}
```
- Dans la fonction `TipTimeLayout()`, mettez à jour l'appel de fonction `calculateTip()`, puis transmettez un paramètre `roundUp` :
 - ```
val tip = calculateTip(amount, tipPercent, roundUp)
```
- Exécutez l'application.

# Calculer un pourboire personnalisé

Ajouter un bouton bascule

- Comme vous pouvez le constater sur les images suivantes, le montant du pourboire est arrondi :



# Calculer un pourboire personnalisé

Ajouter la prise en charge du mode Paysage

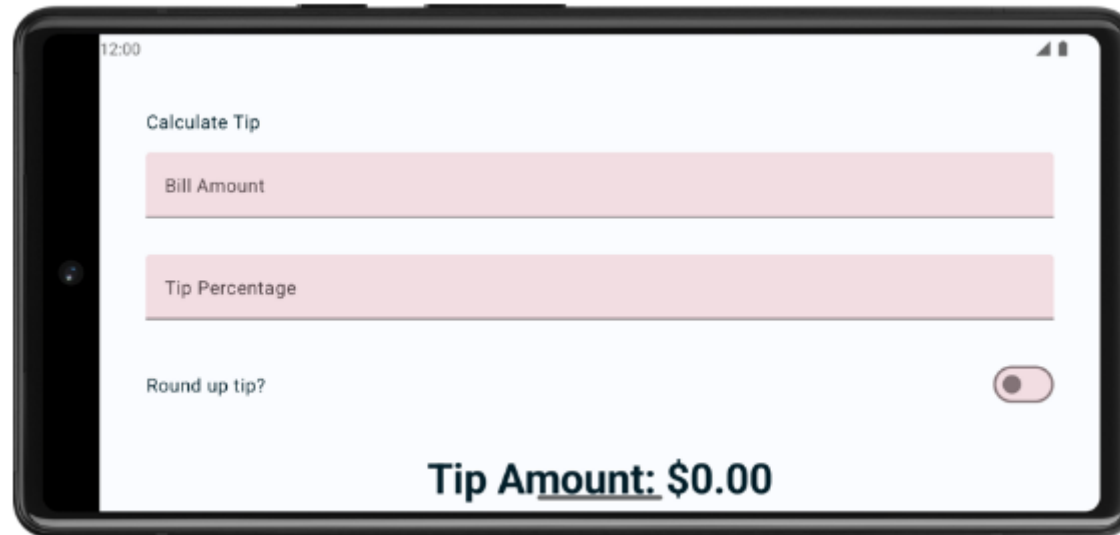
- Les appareils Android sont disponibles sous différentes formes (téléphones, tablettes, appareils pliables et appareils Chrome OS) et proposent un large choix quant à la taille de l'écran.
- Votre application doit être compatible avec les modes Portrait et Paysage.
- Testez votre application en mode Paysage.
- Activez la rotation automatique.
- Faites pivoter l'émulateur ou l'appareil vers la gauche.
- Le montant du pourboire n'est plus visible.



# Calculer un pourboire personnalisé

Ajouter la prise en charge du mode Paysage

- Pour résoudre ce problème, vous avez besoin d'une barre de défilement verticale afin de faire défiler l'écran de votre application.



# Calculer un pourboire personnalisé

Ajouter la prise en charge du mode Paysage

- Ajoutez `.verticalScroll(rememberScrollState())` au modificateur pour ajouter la colonne permettant le défilement vertical de l'écran.
- `rememberScrollState()` crée et mémorise automatiquement l'état de défilement.

- `@Composable`
- `fun TipTimeLayout() {`
- `// ...`
- `Column(`
- `modifier = Modifier`
- `.padding(40.dp)`
- `.verticalScroll(rememberScrollState()),`

# Calculer un pourboire personnalisé

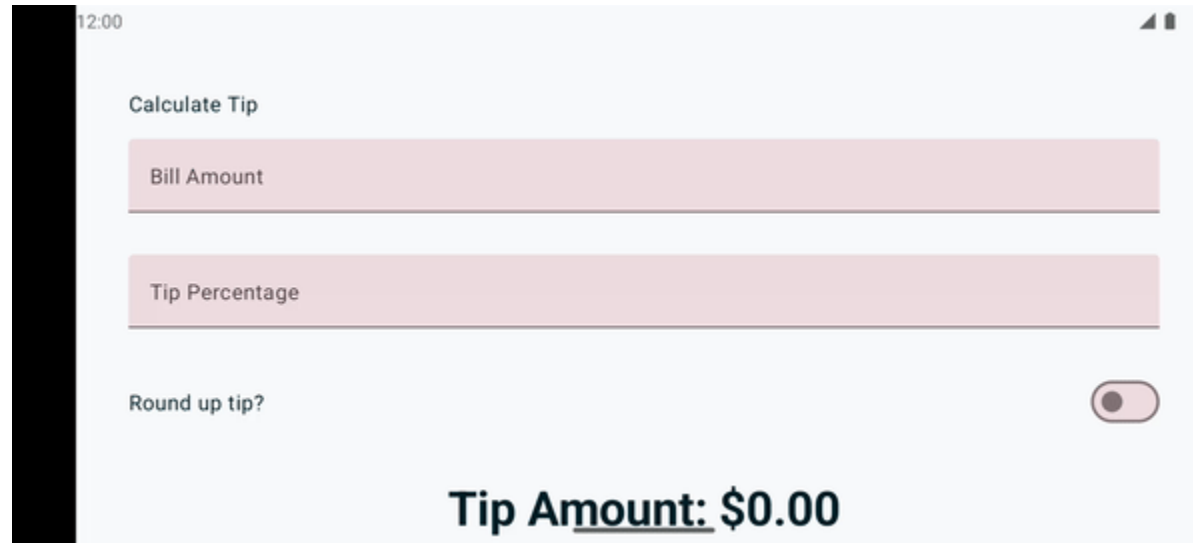
Ajouter la prise en charge du mode Paysage

- `horizontalAlignment = Alignment.CenterHorizontally,`
- `verticalArrangement = Arrangement.Center`
- `) {`
- `//...`
- `}`
- `}`
- Exécutez la commande d'importation suivante :
  - `import androidx.compose.foundation.rememberScrollState`
  - `import androidx.compose.foundation.verticalScroll`
- Exécutez à nouveau l'application.

# Calculer un pourboire personnalisé

Ajouter la prise en charge du mode Paysage

- Essayez de faire défiler l'écran en mode Paysage.



The screenshot shows a mobile application interface for calculating a tip. The status bar at the top displays the time 12:00 and battery level. The app title 'Calculate Tip' is at the top. Below it are two input fields: 'Bill Amount' and 'Tip Percentage', both with pink borders. A toggle switch for 'Round up tip?' is on the right, currently turned off. At the bottom, the text 'Tip Amount: \$0.00' is displayed. A black vertical bar is visible on the left side of the screen.

# Calculer un pourboire personnalisé

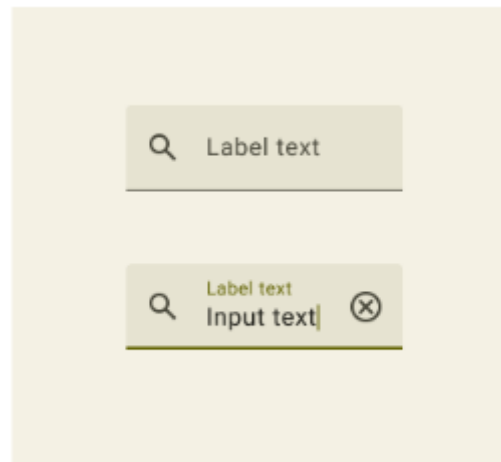
Ajouter une icône précédant les champs de texte

- **Les icônes peuvent rendre le champ de texte plus attrayant et fournir des informations supplémentaires.**
- Les icônes permettent de transmettre des informations sur l'objectif du champ de texte, comme le type de données attendu ou le type d'entrée requis.
- Par exemple, l'icône d'un téléphone placée à côté d'un champ de texte peut indiquer que l'utilisateur doit saisir un numéro de téléphone.
- Les icônes peuvent faciliter la saisie de l'utilisateur en fournissant des indications visuelles sur l'entrée attendue.

# Calculer un pourboire personnalisé

Ajouter une icône précédant les champs de texte

- Par exemple, une icône représentant un calendrier placé à côté d'un champ de texte peut indiquer que l'utilisateur doit saisir une date.
- Vous trouverez ci-dessous un exemple de champ de texte avec une icône de recherche, indiquant que l'utilisateur doit saisir le terme de recherche.



# Calculer un pourboire personnalisé

Ajouter une icône précédant les champs de texte

- Ajoutez un autre paramètre au composable `EditNumberField()`, appelé **leadingIcon**, de type `Int`.
  - Annotez-le avec `@DrawableRes`.
  - `@Composable`
  - `fun EditNumberField(`
  - `@StringRes label: Int,`
  - `@DrawableRes leadingIcon: Int,`
  - `keyboardOptions: KeyboardOptions,`
  - `value: String,`
  - `onValueChanged: (String) -> Unit,`
  - `modifier: Modifier = Modifier`
  - `)`

# Calculer un pourboire personnalisé

Ajouter une icône précédant les champs de texte

- Exécutez la commande d'importation suivante :
  - `import androidx.annotation.DrawableRes`
  - `import androidx.compose.material3.Icon`
- Ajoutez l'icône avant le champ de texte.
- `leadingIcon` accepte un composable. Transmettez le composable `Icon` suivant :
  - `TextField(`
  - `value = value,`
  - `leadingIcon = { Icon(painter = painterResource(id = leadingIcon), null) },`
  - `//...`
  - `)`



# Calculer un pourboire personnalisé

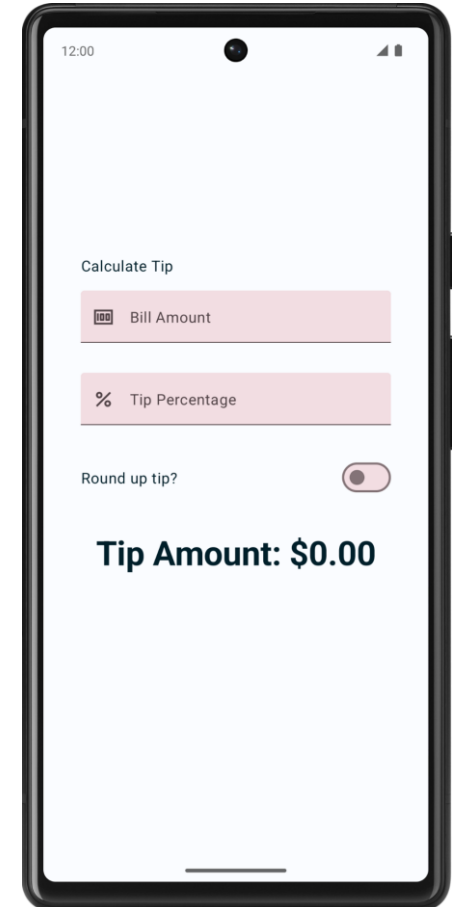
Ajouter une icône précédant les champs de texte

- Transmettez l'icône précédant le texte aux champs de texte.
- Pour plus de commodité, les icônes sont déjà présentes dans le code de démarrage.
  - `EditNumberField(`
  - `label = R.string.bill_amount,`
  - **`leadingIcon = R.drawable.money,`**
  - `// Other arguments`
  - `)`
  - `EditNumberField(`
  - `label = R.string.how_was_the_service,`
  - **`leadingIcon = R.drawable.percent,`**

# Calculer un pourboire personnalisé

Ajouter une icône précédant les champs de texte

- `// Other arguments`
- `)`
- Exécutez l'application.
- Félicitations ! Votre application peut à présent calculer des pourboires personnalisés.



# Écrire des tests automatisés

Cet atelier de programmation vous présente les tests automatisés d'Android et explique en quoi ils permettent de développer des applications évolutives et robustes.

# Écrire des tests automatisés

- Vous découvrirez la différence entre la logique de l'interface utilisateur et la logique métier, et apprendrez à tester les deux.
- Enfin, vous apprendrez à écrire et à exécuter des tests automatisés dans Android Studio.

# Écrire des tests automatisés

## Tests automatisés

- **Les tests automatisés sont des lignes de code qui vérifient qu'un autre élément de code fonctionne correctement.**
- Ils constituent **une partie importante du processus de développement d'applications** car ils permettent de vérifier l'adéquation, le comportement et l'utilisation d'une application avant sa mise à disposition.
- Les tests automatisés sont particulièrement utiles pour tester le code de l'application et les exigences fonctionnelles de l'application elle-même.

# Écrire des tests automatisés

## Tests automatisés

- En créant des tests en parallèle à la création du code de l'application, on réduit la charge de travail et on s'assure que l'application fonctionne correctement sans passer trop de temps à la tester manuellement.
- **En quoi les tests automatisés sont importants**
- Dans un premier temps, vous pourriez avoir l'impression que vous n'avez pas vraiment besoin de tester votre application.
- Détrompez-vous : il s'agit d'une étape indispensable, quelle que soit la taille ou la complexité de votre projet.

# Écrire des tests automatisés

## Tests automatisés

- Pour développer votre base de code, vous devez tester les fonctionnalités existantes au fur et à mesure que vous ajoutez des éléments, ce qui n'est possible que si vous disposez déjà de tests.
- À mesure que votre application se développe, les tests manuels nécessitent beaucoup plus d'efforts que les tests automatisés.
- De plus, **lorsque vous commencez à travailler sur des applications en production, les tests deviennent d'autant plus incontournables**, en particulier lorsque votre base d'utilisateurs est étendue.
- Par exemple, vous devez prendre en compte de nombreux types d'appareils, équipés de diverses versions d'Android.

# Écrire des tests automatisés

## Tests automatisés

- Arrivé à un certain stade, vous atteignez un niveau où les tests automatisés représentent la majorité des scénarios d'utilisation bien plus rapidement que les tests manuels.
- Lorsque vous exécutez des tests avant de publier le nouveau code, vous pouvez modifier le code existant afin d'éviter de publier une application présentant des comportements inattendus.
- N'oubliez pas que **les tests automatisés sont exécutés à l'aide d'un logiciel, par opposition aux tests manuels effectués par une personne qui interagit directement avec un appareil.**



# Écrire des tests automatisés

## Tests automatisés

- Les tests automatisés et manuels jouent un rôle essentiel pour que les utilisateurs bénéficient d'une expérience agréable.
- Toutefois, les tests automatisés peuvent être plus précis et optimiser la productivité de votre équipe, car ils s'exécutent de façon autonome.
- De plus, leur exécution est beaucoup plus rapide que les tests manuels.

# Écrire des tests automatisés

## Tests automatisés

- **Types de tests automatisés**
- **Tests en local**
- Les tests en local testent directement un court extrait de code pour vérifier qu'il fonctionne correctement.
- Vous pouvez ainsi tester des fonctions, des classes et des propriétés.
- Les tests locaux sont exécutés sur votre poste de travail, ce qui signifie qu'ils s'exécutent dans un environnement de développement sans nécessiter d'appareil ni d'émulateur.
- Pour le dire plus simplement, les tests en local sont exécutés sur votre ordinateur.

# Écrire des tests automatisés

## Tests automatisés

- De plus, les frais généraux associés sont très faibles en termes de ressources informatiques, ce qui leur permet de fonctionner rapidement, même avec des ressources limitées.
- Android Studio est conçu pour exécuter automatiquement des tests en local.
- **Tests d'instrumentation**
- Un test d'instrumentation est un test de l'interface utilisateur.
- Les tests d'instrumentation vous permettent de tester des parties d'une application qui dépendent de l'API Android, ainsi que de ses services et API de plate-forme.

# Écrire des tests automatisés

## Tests automatisés

- **Contrairement aux tests en local, les tests d'interface utilisateur lancent une application ou une partie d'une application, simulent les interactions des utilisateurs et vérifient que la réaction est correcte.**
- **Lorsque vous exécutez un test d'instrumentation sous Android, le code de test est en réalité intégré à son propre Android Application Package (APK), comme une application Android standard.**
- Un APK est un fichier compressé contenant l'ensemble du code et des fichiers nécessaires à l'exécution de l'application sur un appareil ou un émulateur.

# Écrire des tests automatisés

## Tests automatisés

- L'APK de test est installé sur l'appareil ou l'émulateur avec l'APK d'application standard.
- L'APK de test effectue ensuite ses tests par rapport à l'APK de l'application.

# Écrire des tests automatisés

## Écrire un test en local

- **Préparer le code de l'application**
- Les tests en local testent directement des méthodes à partir du code de l'application.
- Le test en local de l'extrait de code suivant vérifie que la méthode `calculateTip()` fonctionne correctement, mais `calculateTip()` est actuellement privée (`private`) et n'est donc pas accessible à partir du test.
- Supprimez la désignation `private` et changez-la en `internal` (interne) :

# Écrire des tests automatisés

## Écrire un test en local

- MainActivity.kt

- **internal** fun calculateTip(amount: Double, tipPercent: Double = 15.0, roundUp: Boolean): String {
- var tip = tipPercent / 100 \* amount
- if (roundUp) {
- tip = kotlin.math.ceil(tip)
- }
- return NumberFormat.getCurrencyInstance().format(tip)
- }

- Dans le fichier MainActivity.kt, sur la ligne située avant la méthode calculateTip(), ajoutez l'annotation @VisibleForTesting :

# Écrire des tests automatisés

## Écrire un test en local

- **@VisibleForTesting**
- internal fun calculateTip(amount: Double, tipPercent: Double = 15.0, roundUp: Boolean): String {
- var tip = tipPercent / 100 \* amount
- if (roundUp) {
- tip = kotlin.math.ceil(tip)
- }
- return NumberFormat.getCurrencyInstance().format(tip)
- }
- La méthode est rendue publique, mais elle indique qu'elle n'est publique qu'à des fins de test.



# Écrire des tests automatisés

## Écrire un test en local

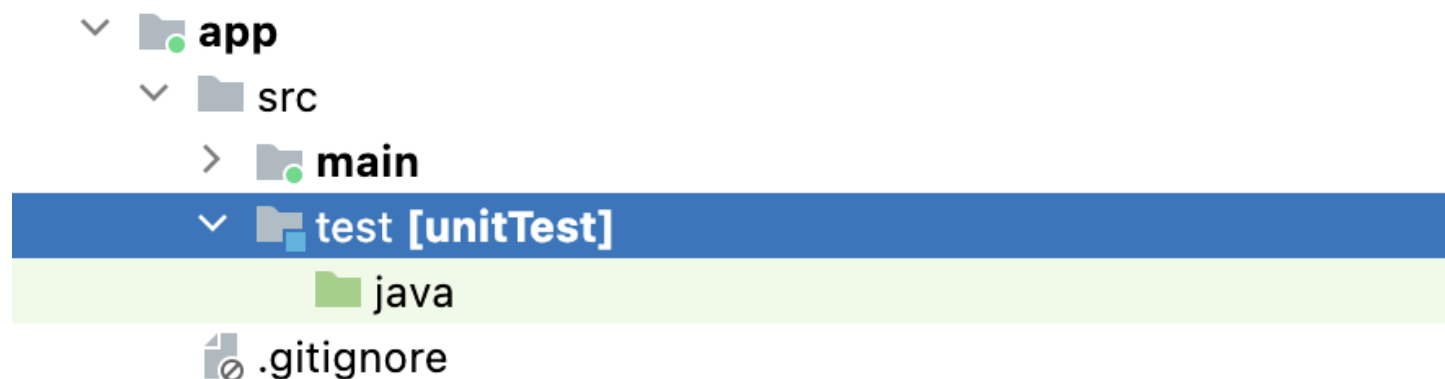
- **Créer le répertoire de tests**

- Dans les projets Android, le répertoire test est l'endroit où les tests locaux sont écrits.
- Créez le répertoire de tests :
- Dans l'onglet Project (Projet), définissez la vue sur "Project" (Projet).
- Effectuez un clic droit sur le répertoire src.
- Sélectionnez New (Nouveau).
- Sélectionnez Directory (Répertoire).

# Écrire des tests automatisés

## Écrire un test en local

- Dans la fenêtre New Directory (Nouveau répertoire), sélectionnez **test/java**.
- Appuyez sur la touche Retour ou Entrée du clavier.
- Le répertoire de tests s'affiche désormais dans l'onglet Project (Projet).



# Écrire des tests automatisés

## Écrire un test en local

- **Le répertoire de tests nécessite une structure de packages identique à celle du répertoire main, qui contient le code de votre appli.**
- En d'autres termes, comme votre code d'application est écrit dans le package main > java > com > example > tiptime, vos tests locaux sont écrits dans test > java > com > example > tiptime.
- Créez cette structure de packages dans le répertoire de tests :
  - Effectuez un clic droit sur le répertoire test/java, puis sélectionnez New > Package (Nouveau > Package).
  - Dans la fenêtre New Package (Nouveau package), saisissez com.example.tiptime.

# Écrire des tests automatisés

## Écrire un test en local

- **Créer la classe de test**
- Maintenant que le package de test est prêt, il est temps d'écrire des tests.
- Commencez par créer la classe de test.
  - Dans l'onglet Project (Projet), cliquez sur app > src > test, puis sur la flèche de développement à côté du répertoire test.
  - Effectuez un clic droit sur le répertoire tiptime, puis sélectionnez New > Kotlin Class/File (Nouveau > Classe/Fichier Kotlin).
  - Saisissez TipCalculatorTests comme nom de classe.

# Écrire des tests automatisés

## Écrire un test en local

- **Écrire le test**
- Comme indiqué précédemment, les tests en local sont utilisés pour tester de courts extraits de code dans l'application.
- La fonction principale de l'application Tip Time est de calculer des pourboires, le test en local doit donc s'assurer que la logique de calcul fonctionne correctement.
- Pour ce faire, vous devez appeler directement la fonction `calculateTip()`, comme vous l'avez fait dans le code de l'application.

# Écrire des tests automatisés

## Écrire un test en local

- Ensuite, vérifiez que la valeur renvoyée correspond au résultat attendu par rapport aux valeurs que vous avez transmises à la fonction.
- Voici quelques points à garder à l'esprit concernant l'écriture de tests automatisés.
- La liste de concepts suivante s'applique aux tests en local, mais aussi aux tests instrumentation.
- À première vue, ils peuvent sembler abstraits, mais d'ici la fin de cet atelier de programmation, vous aurez eu l'occasion de vous familiariser avec eux.

# Écrire des tests automatisés

## Écrire un test en local

- **Rédigez vos tests automatisés sous la forme de méthodes.**
  - Annotez la méthode avec **@Test**. De cette façon, le compilateur sait qu'il s'agit d'une méthode de test et l'exécute en conséquence.
- Remarque : L'annotation **@Test** est importée à partir du package **org.junit.Test** avec l'instruction d'importation **import org.junit.Test**.
- Assurez-vous que le nom décrit clairement l'objet du test et le résultat attendu.
- Les méthodes de test n'utilisent pas la même logique que les méthodes d'application standards.
- Elles ne se préoccupent pas de la mise en œuvre.

# Écrire des tests automatisés

## Écrire un test en local

- Elles se contentent de vérifier le résultat pour une entrée donnée.
- Autrement dit, **les méthodes de test n'exécutent qu'un ensemble d'instructions pour vérifier que l'interface utilisateur ou la logique d'une application fonctionne correctement.**
- **Les tests se terminent généralement par une assertion** qui permet de vérifier qu'une condition donnée est remplie.
- Les assertions se présentent sous la forme d'un appel de méthode dont le nom contient **assert**.
- Par exemple, l'assertion **assertTrue()** est communément utilisée dans les tests Android.



# Écrire des tests automatisés

## Écrire un test en local

- Les instructions d'assertion sont utilisées dans la plupart des tests, mais ce n'est que rarement le cas dans du code d'application.
- Écrivez le test :
  - Créez une méthode pour tester le calcul d'un pourboire de 20 % pour un montant de 10 \$. Le résultat attendu de ce calcul est 2 \$.
  - `import org.junit.Test`
  - `class TipCalculatorTests {`
  - `@Test`
  - `fun calculateTip_20PercentNoRoundup() {`
  - `}`
  - `}`

# Écrire des tests automatisés

## Écrire un test en local

- N'oubliez pas que la méthode `calculateTip()` du fichier `MainActivity.kt` dans le code de l'application nécessite trois paramètres : le montant de la facture, le pourcentage du pourboire et un indicateur permettant d'arrondir ou non le résultat.
  - `fun calculateTip(amount: Double, tipPercent: Double, roundUp: Boolean)`
- Lorsque cette méthode doit être appelée à partir du test, ces paramètres doivent être transmis de la même façon que dans le code de l'application.
- Dans la méthode `calculateTip_20PercentNoRoundup()`, créez deux variables constantes : une variable `amount` définie sur la valeur 10.00 et une variable `tipPercent` définie sur la valeur 20.00.

# Écrire des tests automatisés

## Écrire un test en local

- `val amount = 10.00`
- `val tipPercent = 20.00`
- Dans le fichier MainActivity.kt du code de l'application, observez le code suivant.
- Le montant du pourboire est mis en forme en fonction des paramètres régionaux de l'appareil.
- MainActivity.kt
  - `...`
  - `NumberFormat.getCurrencyInstance().format(tip)`
  - `...`

# Écrire des tests automatisés

## Écrire un test en local

- La même mise en forme doit être utilisée lors de la vérification du montant du pourboire attendu dans le test.
- Créez une variable `expectedTip` définie sur `NumberFormat.getCurrencyInstance().format(2)`.
- La variable `expectedTip` est ensuite comparée au résultat de la méthode `calculateTip()`.
- C'est de cette façon que le test garantit le bon fonctionnement de la méthode.
- À l'étape précédente, vous avez défini la variable `amount` sur une valeur de 10.00 et la variable `tipPercent` sur une valeur de 20.00.

# Écrire des tests automatisés

## Écrire un test en local

- Vingt pour cent de 10 \$ font 2 \$, d'où la variable `expectedTip` définie sur une devise mise en forme avec une valeur de 2.
- Gardez à l'esprit que la méthode `calculateTip()` renvoie une valeur `String` mise en forme.
- Appelez la méthode `calculateTip()` avec les variables `amount` et `tipPercent`, puis transmettez un argument `false` pour l'arrondi.
- Dans le cas présent, vous n'avez pas besoin de vous soucier d'arrondir, car le résultat attendu n'est pas concerné par les arrondis.
- Stockez le résultat de l'appel de méthode dans une variable `actualTip` constante.

# Écrire des tests automatisés

## Écrire un test en local

- Jusqu'à présent, l'écriture de ce test ne diffère pas beaucoup de celle d'une méthode standard dans le code d'une application.
- Toutefois, maintenant que vous disposez de la valeur renvoyée par la méthode que vous souhaitez tester, vous devez déterminer si cette valeur est correcte en utilisant une assertion.
- En règle générale, les assertions sont l'objectif final d'un test automatisé, ce pour quoi elles ne sont pas communément utilisées dans du code d'application réel.
- Ici, vous devez vous assurer que la variable `actualTip` correspond à la variable `expectedTip`.

# Écrire des tests automatisés

## Écrire un test en local

- Pour ce faire, vous pouvez utiliser la méthode **assertEquals()** de la bibliothèque **JUnit**.
- La méthode assertEquals() prend deux paramètres en entrée : une valeur attendue et une valeur réelle.
- Si ces valeurs sont égales, l'assertion et le test sont concluants.
- Dans le cas contraire, l'assertion et le test échouent.
- Appelez la méthode assertEquals(), puis transmettez les variables expectedTip et actualTip en tant que paramètres :
  - `import org.junit.Assert.assertEquals`
  - `import org.junit.Test`

# Écrire des tests automatisés

## Écrire un test en local

- `import java.text.NumberFormat`
- `class TipCalculatorTests {`
- `@Test`
- `fun calculateTip_20PercentNoRoundup() {`
- `val amount = 10.00`
- `val tipPercent = 20.00`
- `val expectedTip = NumberFormat.getCurrencyInstance().format(2)`
- `val actualTip = calculateTip(amount = amount, tipPercent = tipPercent,`
- `false)`
- `assertEquals(expectedTip, actualTip)`
- `}`
- `}`



# Écrire des tests automatisés

## Écrire un test en local

- Remarque : La bibliothèque JUnit comporte de nombreuses assertions. Vous pouvez trouver ci-dessous une liste d'assertions communément utilisées.
  - assertEquals()
  - assertNotEquals()
  - assertTrue()
  - assertFalse()
  - assertNull()
  - assertNotNull()
  - assertThat()
- Pour en savoir plus, consultez la section [Assertions](#).

# Écrire des tests automatisés

## Écrire un test en local

- **Exécuter le test**
- Vous pouvez maintenant exécuter votre test !
- Vous avez peut-être remarqué que des flèches apparaissent à côté du numéro de ligne du nom de classe et de la fonction de test.
- Vous pouvez cliquer sur ces flèches pour lancer le test.
- Lorsque vous cliquez sur la flèche située à côté d'une méthode, vous n'exécutez que la méthode de test en question.
- Si une classe comporte plusieurs méthodes de test, vous pouvez cliquer sur la flèche située à côté de la classe pour lancer toutes les méthodes de test qu'elle contient.

# Écrire des tests automatisés

## Écrire un test en local

- Exécutez le test :
  - Cliquez sur les flèches à côté de la déclaration de la classe, puis sur Run 'TipCalculatorTests' (Exécuter 'TipCalculatorTests').
- Voici ce qu'il devrait de passer :
  - Dans la marge, les flèches sont remplacées par une coche et un triangle verts. Cela indique que le test a réussi.



```
23    class TipCalculatorTests {
24 @Test
25    fun calculateTip_20PercentNoRoundup() {
26 val amount = 10.00
27 val tipPercent = 20.00
```

# Écrire des tests automatisés

## Écrire un test en local

- La sortie s'affiche en bas du volet Run (Exécuter).
- Une indication que les tests ont réussi devrait s'afficher.

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- Créer le **répertoire d'instrumentation**
- Le répertoire d'instrumentation est créé de la même manière que le répertoire de tests local.
- Effectuez un clic droit sur le répertoire src, puis sélectionnez New > Directory (Nouveau > Répertoire).
- L'option "Directory" (Répertoire) est sélectionnée dans le menu
- Dans la fenêtre New Directory (Nouveau répertoire), sélectionnez **androidTest/java**.

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- Appuyez sur la touche Retour ou Entrée du clavier.
- Le répertoire androidTest s'affiche désormais dans l'onglet Project (Projet).
- Tout comme les répertoires main et test ont la même structure de packages, le répertoire androidTest doit contenir cette même structure.
- Effectuez un clic droit sur le dossier androidTest/java, puis sélectionnez New > Package (Nouveau > Package).
- Dans la fenêtre New Package (Nouveau package), saisissez com.example.tiptime.

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- Appuyez sur la touche Retour ou Entrée du clavier.
- La structure complète des packages du répertoire androidTest est maintenant visible dans l'onglet Project (Projet).
- **Créer la classe de test**
- **Dans les projets Android, le répertoire de test d'instrumentation est androidTest.**
- Pour créer un test d'instrumentation, vous devez répéter le processus que vous avez utilisé pour créer un test local, mais cette fois-ci dans le répertoire androidTest.

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- Créez la classe de test :
  - Dans le volet du projet, accédez au répertoire androidTest.
  - Cliquez sur les flèches de développement à côté de chaque répertoire jusqu'à trouver tiptime.
- Effectuez un clic droit sur le répertoire tiptime, puis sélectionnez New > Kotlin Class/File (Nouveau > Classe/Fichier Kotlin).
- Saisissez TipUITests comme nom de classe.
- **Écrire le test**
- **Le code d'un test d'instrumentation est assez différent du code d'un test en local.**



# Écrire des tests automatisés

## Écrire un test d'instrumentation

- **Les tests d'instrumentation testent une instance réelle de l'application et de son interface utilisateur.**
- Par conséquent, **le contenu de l'UI doit être défini de la même manière que vous l'avez fait avec la méthode onCreate() du fichier MainActivity.kt** lorsque vous avez écrit le code de l'application Tip Time.
- Il s'agit d'un prérequis à l'écriture de tous les tests d'instrumentation pour les applications créées avec Compose.

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- Dans le cas des tests de l'application Tip Time, vous allez rédiger des instructions pour interagir avec les composants de l'UI, de sorte que le processus de calcul des pourboires soit testé via l'interface utilisateur.
- Le concept d'un test d'instrumentation peut sembler abstrait au début.
- Nous allons détailler ce processus au cours des étapes suivantes.
- **Écrivez le test :**
- Créez une variable `composeTestRule` définie sur le résultat de la méthode `createComposeRule()`, puis annotez-la avec `Rule` :

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- `import androidx.compose.ui.test.junit4.createComposeRule`
- `import org.junit.Rule`
- `class TipUITests {`
- `@get:Rule`
- `val composeTestRule = createComposeRule()`
- `}`
- Créez une méthode `calculate_20_percent_tip()` et annotez-la avec `@Test` :
- `import org.junit.Test`
- `@Test`
- `fun calculate_20_percent_tip() {`
- `}`

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- Le compilateur sait que les méthodes annotées avec `@Test` dans le répertoire `androidTest` font référence à des tests d'instrumentation et que les méthodes annotées avec `@Test` dans le répertoire `test` font référence à des tests en local.
- Dans le corps de la fonction, appelez `composeTestRule.setContent()`.
- Cela définit le contenu de l'interface utilisateur de `composeTestRule`.
- Dans le corps du lambda de la fonction, appelez `TipTimeTheme()` avec un corps de lambda qui appelle `TipTimeLayout()`.

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- `import com.example.tiptime.ui.theme.TipTimeTheme`
- `@Test`
- `fun calculate_20_percent_tip() {`
- `composeTestRule.setContent {`
- `TipTimeTheme {`
- `TipTimeLayout()`
- `}`
- `}`
- `}`

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- **Lorsque vous avez terminé, le code doit ressembler à celui écrit pour définir le contenu de la méthode onCreate() dans le fichier MainActivity.kt.**
- Maintenant que le contenu de l'UI est configuré, vous pouvez rédiger des instructions pour interagir avec les composants de l'interface de l'application.
- Dans cette application, vous devez **vérifier qu'elle affiche la valeur correcte du pourboire en fonction du montant de la facture et des pourcentages de pourboire saisis.**

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- Les composants d'interface utilisateur sont accessibles en tant que nœuds via `composeTestRule`.
- Pour ce faire, une technique communément utilisée consiste à accéder à un nœud contenant un texte particulier à l'aide de la méthode **`onNodeWithText()`**.
- Utilisez la méthode `onNodeWithText()` afin d'accéder au composable `TextField` pour le montant de la facture :
  - `import androidx.compose.ui.test.onNodeWithText`
  - `@Test`
  - `fun calculate_20_percent_tip() {`

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- `composeTestRule.setContent {`
  - `TipTimeTheme {`
  - `TipTimeLayout()`
  - `}`
  - `}`
  - `composeTestRule.onNodeWithText("Bill Amount")`
  - `}`
- Vous pouvez ensuite appeler la méthode **performTextInput()** et transmettre le texte que vous souhaitez saisir pour remplir le composable TextField.



# Écrire des tests automatisés

## Écrire un test d'instrumentation

- Dans le champ TextField, indiquez le montant de la facture avec une valeur 10 :

- `import androidx.compose.ui.test.performTextInput`
- `@Test`
- `fun calculate_20_percent_tip() {`
- `composeTestRule.setContent {`
- `TipTimeTheme {`
- `TipTimeLayout()`
- `}`
- `}`
- `composeTestRule.onNodeWithText("Bill Amount")`

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- `.performTextInput("10")`
- `}`
- Utilisez la même approche pour renseigner le champ `OutlinedTextField` avec le pourcentage de pourboire, en l'occurrence une valeur de 20 :
  - `@Test`
  - `fun calculate_20_percent_tip() {`
  - `composeTestRule.setContent {`
  - `TipTimeTheme {`
  - `TipTimeLayout()`
  - `}`

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- `}`
- `composeTestRule.onNodeWithText("Bill Amount")`
- `.performTextInput("10")`
- `composeTestRule.onNodeWithText("Tip Percentage").performTextInput("20")`
- `}`
- Une fois que tous les composables TextField ont été renseignés, le pourboire s'affiche dans un composable Text en bas de l'écran dans l'application.

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- Maintenant que vous avez demandé au test d'insérer ces composables TextField, vous devez vous assurer que le composable Text affiche le bon pourboire à l'aide d'une assertion.
- **Dans les tests d'instrumentation avec Compose, les assertions peuvent être appelées directement sur les composants d'UI.**
- De nombreuses assertions sont disponibles, mais dans le cas présent, vous allez utiliser la méthode **assertExists()**.
- Le composable Text, qui affiche le montant du pourboire, doit s'afficher ainsi : Tip Amount: \$2.00.

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- Remarque : Le symbole de la devise peut varier en fonction des paramètres régionaux de l'appareil. L'utilisation de "\$" n'est qu'un exemple. La devise doit être mise en forme en fonction des paramètres régionaux.
- Créez une assertion pour vérifier qu'un nœud contenant ce texte existe :
  - `import java.text.NumberFormat`
  - `@Test`
  - `fun calculate_20_percent_tip() {`
  - `composeTestRule.setContent {`
  - `TipTimeTheme {`

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- `Surface (modifier = Modifier.fillMaxSize()){`
- `TipTimeLayout()`
- `}`
- `}`
- `}`
- `composeTestRule.onNodeWithText("Bill Amount")`
- `.performTextInput("10")`
- `composeTestRule.onNodeWithText("Tip Percentage").performTextInput("20")`
- **`val expectedTip = NumberFormat.getCurrencyInstance().format(2)`**
- **`composeTestRule.onNodeWithText("Tip Amount`**

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- `: $expectedTip").assertExists(`
- `"No node with this text was found."`
- `)`
- `}`
- Exécuter le test
- Le processus d'exécution d'un test d'instrumentation est identique à celui d'un test en local.
- Vous pouvez cliquer sur les flèches dans la marge, en face de chaque déclaration, pour exécuter un test individuel ou l'ensemble de la classe de test.

# Écrire des tests automatisés

## Écrire un test d'instrumentation

- Cliquez sur les flèches en face de la déclaration de classe.
- Vous pouvez voir les tests s'exécuter sur votre appareil ou émulateur.